

EEG based classification of Digit-Span Memorization vs. Listening

[YouTube Video](#)

Acknowledgements

This data has been downloaded from:

Yuri G. Pavlov, Dauren Kasanov, Alexandra I. Kosachenko, and Alexander I. Kotyusov (2024). EEG, pupillometry, ECG and photoplethysmography, and behavioral data in the digit span task and rest. OpenNeuro. [Dataset] doi: [doi:10.18112/openneuro.ds003838.v1.0.6](https://doi.org/10.18112/openneuro.ds003838.v1.0.6)

<https://openneuro.org/datasets/ds003838/versions/1.0.6>

Import libraries

```
import mne
import pandas as pd
import csv
import matplotlib.pyplot as plt
import warnings
import os
import numpy as np
from mne_icalabel import label_components
from IPython.display import display
import traceback
from sklearn.model_selection import RandomizedSearchCV,
LeaveOneGroupOut
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, ConfusionMatrixDisplay
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.dummy import DummyClassifier

mne.set_log_level("ERROR")
warnings.filterwarnings("ignore", category=UserWarning,
module="pymatreader")

c:\Users\atifm\AppData\Local\Programs\Python\Python313\Lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please
update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

Intro

"This dataset consists of raw 64-channel EEG, cardiovascular (electrocardiography and photoplethysmography), and pupillometry data from 86 human participants during 4 minutes of eyes-closed resting and during performance of a classic working memory task – digit span task with serial recall. The participants either memorized (memory) or just listened to (control condition) sequences of 5, 9, or 13 digits presented auditorily with 2 second stimulus onset asynchrony. The dataset can be used for (1) developing algorithms for cognitive load discrimination and detection of cognitive overload; (2) studying neural (event-related potentials and brain oscillations) and peripheral physiological (electrocardiography, photoplethysmography, and pupillometry) signals during encoding and maintenance of each sequentially presented memory item in a fine time scale; (3) correlating cognitive load and individual differences in working memory to neural and peripheral physiology, and studying the relationship between the physiological signals; (4) integration of the physiological findings with the vast knowledge coming from behavioral studies of verbal working memory in simple span paradigms.

EEG, pupillometry, ECG and photoplethysmography, and behavioral data are stored separately in corresponding folders. Each data record can consist of four data folders: beh - behavioral data: correctness of the recall in the memory trials ecg - electrocardiography (ECG) and photoplethysmography (PPG) data eeg - EEG data pupil - pupillometry and eye-tracking data

Some of the participants had some physiological data missing: sub-017, sub-094 have no pupillometry data sub-017, sub-037, sub-066 have no ECG and PPG data sub-013, sub-014, sub-015, sub-016, sub-017, sub-018, sub-019, sub-020, sub-021, sub-022, sub-023, sub-024, sub-025, sub-026, sub-027, sub-028, sub-029, sub-030, sub-031, sub-037, sub-066 have no EEG data"

Yuri G. Pavlov, Dauren Kasanov, Alexandra I. Kosachenko, and Alexander I. Kotyusov (2024). EEG, pupillometry, ECG and photoplethysmography, and behavioral data in the digit span task and rest. OpenNeuro. [Dataset] doi: [doi:10.18112/openneuro.ds003838.v1.0.6](https://doi.org/10.18112/openneuro.ds003838.v1.0.6)

EDA

EDA code

```
# The EEG data is in the `_eeg.set` files
participants = [32, 33, 34, 35, 36, 38, 39, 40, 41, 42, 43, 44, 45,
46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62,
63, 64, 65]
eeg_eda_df = []

for participant in participants:
    file = f"data/sub-0{participant}/eeg/sub-0{participant}_task-
memory_eeg.set"
    if os.path.exists(file):
        raw = mne.io.read_raw_eeglab(file, preload=True)
        eeg_eda = {"participant" : {participant}, "channels":
raw.ch_names, "num_channels" : {len(raw.ch_names)}, "num_readings" :
```

```

raw.n_times, "frequency" : raw.info["sfreq"]}]
    eeg_eda_df.append(eeg_eda)
else:
    print(f"ERROR: EEG file DOES NOT EXIST for participant
{participant}!")

eeg_eda_df_summary = pd.DataFrame(eeg_eda_df)
display(eeg_eda_df_summary)
display(eeg_eda_df_summary.describe(include="all"))
print(eeg_eda_df_summary["channels"][0])

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"participant","rawType":"object","type":"unknown"},
{"name":"channels","rawType":"object","type":"unknown"},
{"name":"num_channels","rawType":"object","type":"unknown"},
{"name":"num_readings","rawType":"int64","type":"integer"},
{"name":"frequency","rawType":"float64","type":"float"}],"ref":"f740d1
2e-90bd-41a7-ae1e-6fa8192f82fe","rows":[["0","{32}"],["Fp1','Fz',
'F3','F7','FT9','FC5','FC1','C3','T7','TP9','CP5','CP1',
'Pz','P3','P7','O1','Oz','O2','P4','P8','TP10','CP6','CP2',
'Cz','C4','T8','FT10','FC6','FC2','F4','F8','Fp2','AF7',
'AF3','AFz','F1','F5','FT7','FC3','C1','C5','TP7','CP3',
'P1','P5','P07','P03','P0z','P04','P08','P6','P2','CPz',
'CP4','TP8','C6','C2','FC4','FT8','F6','AF8','AF4',
'F2']","{63}","7395320","1000.0"],["1","{33}"],["Fp1','Fz','F3',
'F7','FT9','FC5','FC1','C3','T7','TP9','CP5','CP1','Pz',
'P3','P7','O1','Oz','O2','P4','P8','TP10','CP6','CP2','Cz',
'C4','T8','FT10','FC6','FC2','F4','F8','Fp2','AF7','AF3',
'AFz','F1','F5','FT7','FC3','C1','C5','TP7','CP3','P1','P5',
'P07','P03','P0z','P04','P08','P6','P2','CPz','CP4','TP8',
'C6','C2','FC4','FT8','F6','AF8','AF4',
'F2']","{63}","8446880","1000.0"],["2","{34}"],["Fp1','Fz','F3',
'F7','FT9','FC5','FC1','C3','T7','TP9','CP5','CP1','Pz',
'P3','P7','O1','Oz','O2','P4','P8','TP10','CP6','CP2','Cz',
'C4','T8','FT10','FC6','FC2','F4','F8','Fp2','AF7','AF3',
'AFz','F1','F5','FT7','FC3','C1','C5','TP7','CP3','P1','P5',
'P07','P03','P0z','P04','P08','P6','P2','CPz','CP4','TP8',
'C6','C2','FC4','FT8','F6','AF8','AF4',
'F2']","{63}","7084360","1000.0"],["3","{35}"],["Fp1','Fz','F3',
'F7','FT9','FC5','FC1','C3','T7','TP9','CP5','CP1','Pz',
'P3','P7','O1','Oz','O2','P4','P8','TP10','CP6','CP2','Cz',
'C4','T8','FT10','FC6','FC2','F4','F8','Fp2','AF7','AF3',
'AFz','F1','F5','FT7','FC3','C1','C5','TP7','CP3','P1','P5',
'P07','P03','P0z','P04','P08','P6','P2','CPz','CP4','TP8',
'C6','C2','FC4','FT8','F6','AF8','AF4',
'F2']","{63}","7342940","1000.0"],["4","{36}"],["Fp1','Fz','F3',
'F7','FT9','FC5','FC1','C3','T7','TP9','CP5','CP1','Pz',
'P3','P7','O1','Oz','O2','P4','P8','TP10','CP6','CP2','Cz',
'C4','T8','FT10','FC6','FC2','F4','F8','Fp2','AF7','AF3',
'AFz','F1','F5','FT7','FC3','C1','C5','TP7','CP3','P1','P5',

```

'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7382180", "1000.0"], ["5", "{38}", ["Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "6951940", "1000.0"], ["6", "{39}", ["Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7182840", "1000.0"], ["7", "{40}", ["Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7176340", "1000.0"], ["8", "{41}", ["Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "8204060", "1000.0"], ["9", "{42}", ["Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "9603300", "1000.0"], ["10", "{43}", ["Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7314740", "1000.0"], ["11", "{44}", ["Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',

'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7409560", "1000.0"], ["12", "{45}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "8374020", "1000.0"], ["13", "{46}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7574380", "1000.0"], ["14", "{47}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7927420", "1000.0"], ["15", "{48}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7386780", "1000.0"], ["16", "{49}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7982500", "1000.0"], ["17", "{50}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7853620", "1000.0"], ["18", "{51}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',

'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7591840", "1000.0"], ["19", "{52}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7222700", "1000.0"], ["20", "{53}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7512320", "1000.0"], ["21", "{54}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7624200", "1000.0"], ["22", "{55}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7288420", "1000.0"], ["23", "{56}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7079980", "1000.0"], ["24", "{57}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7443940", "1000.0"], ["25", "{58}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',

'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7561940", "1000.0"], ["26", "{59}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "8106780", "1000.0"], ["27", "{60}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7484940", "1000.0"], ["28", "{61}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "8328800", "1000.0"], ["29", "{62}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7493960", "1000.0"], ["30", "{63}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "8302620", "1000.0"], ["31", "{64}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7355760", "1000.0"], ["32", "{65}", ["'Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', '01', '0z', '02', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',


```
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", "7057880", "1000.0"]], "shape": {"columns": 5, "rows": 33}}

{"columns": [{"name": "index", "rawType": "str", "type": "string"},
{"name": "participant", "rawType": "object", "type": "unknown"},
{"name": "channels", "rawType": "object", "type": "unknown"},
{"name": "num_channels", "rawType": "object", "type": "unknown"},
{"name": "num_readings", "rawType": "float64", "type": "float"},
{"name": "frequency", "rawType": "float64", "type": "float"}], "ref": "8e9292
0a-3d62-4768-9cdb-884df52f6bb0", "rows":
[["count", "33", "33", "33", "33.0", "33.0"],
["unique", "33", "1", "1", null, null], ["top", "{32}", ["Fp1', 'Fz', 'F3',
'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz',
'P3', 'P7', 'O1', 'Oz', 'O2', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz',
'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3',
'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8',
'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4',
'F2']", "{63}", null, null], ["freq", "1", "33", "33", null, null],
["mean", null, null, null, "7637856.363636363", "1000.0"],
["std", null, null, null, "545641.5632412579", "0.0"],
["min", null, null, null, "6951940.0", "1000.0"],
["25%", null, null, null, "7314740.0", "1000.0"],
["50%", null, null, null, "7484940.0", "1000.0"],
["75%", null, null, null, "7927420.0", "1000.0"],
["max", null, null, null, "9603300.0", "1000.0"]], "shape":
{"columns": 5, "rows": 11}}

['Fp1', 'Fz', 'F3', 'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9',
'CP5', 'CP1', 'Pz', 'P3', 'P7', 'O1', 'Oz', 'O2', 'P4', 'P8', 'TP10',
'CP6', 'CP2', 'Cz', 'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8',
'Fp2', 'AF7', 'AF3', 'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5',
'TP7', 'CP3', 'P1', 'P5', 'P07', 'P03', 'P0z', 'P04', 'P08', 'P6',
'P2', 'CPz', 'CP4', 'TP8', 'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8',
'AF4', 'F2']
```

EDA conclusion

- We will use 33 participants for our analysis
- Each participant has a 64-channel EEG recording (we have 63 channels because one electrode is a reference)
- The EEG data is recorded at 1000Hz
- The number of EEG data points is between 6951940.0 and 9603300.0 (mean = 7637856.363636363)
- The EEG channels are ['Fp1', 'Fz', 'F3', 'F7', 'FT9', 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz', 'P3', 'P7', 'O1', 'Oz', 'O2', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz', 'C4', 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3', 'AFz', 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5', 'P07', 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8', 'C6', 'C2', 'FC4', 'FT8', 'F6', 'AF8', 'AF4', 'F2']


```
'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8', 'C6', 'C2',  
'FC4', 'FT8', 'F6', 'AF8', 'AF4', 'F2']
```

- The set of electrodes is consistent across participants

EEG pre-processing

Load, filter, and re-reference data

```
participants = [32, 33, 34, 35, 36, 38, 39, 40, 41, 42, 43, 44, 45,  
46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62,  
63, 64, 65]  
print(f"N = {len(participants)}")  
  
cache_dir = "data/filtered-referenced-eeg"  
os.makedirs(cache_dir, exist_ok=True)  
  
for participant in participants:  
    cached_file = f"{cache_dir}/sub-0{participant}-task-eeg_filtered-  
referenced.fif"  
  
    if os.path.exists(cached_file):  
        print("File exists!")  
    else:  
        # Cached, pre-processed file not found: load raw file  
        file = f"data/sub-0{participant}/eeg/sub-0{participant}_task-  
memory_eeg.set"  
        raw = mne.io.read_raw_eeglab(file, preload=True)  
  
        # Frequency filter: High-pass 1Hz, Low-pass 45Hz & re-  
        reference to averaged reference  
        # Based on the original authors' implementation: (Kosachenko  
        et al., 2023)  
        raw.filter(l_freq=1, h_freq=45)  
        raw.set_eeg_reference("average")  
  
        # Save file to cache  
        raw.save(cached_file, overwrite=True)
```

ICA and epoching

```
participant = 40  
cache_dir = "data/filtered-referenced-eeg"  
cached_file = f"{cache_dir}/sub-0{participant}-task-eeg_filtered-  
referenced.fif"  
  
if os.path.exists(cached_file):  
    print(f"Found file {cached_file}")  
    raw = mne.io.read_raw_fif(cached_file, preload=True)  
    ica = mne.preprocessing.ICA(n_components=0.99, method="fastica",  
    random_state=42)
```

```

        ica.fit(raw)
        ica.plot_components()
        ica_labels = label_components(raw, ica, "iclabel")
        display(ica_labels)
    else:
        print(f"File - {cached_file} not found!")

display(pd.DataFrame(ica_labels))

```

For each participant remove the artifacts using mne-icalabel

```

cache_dir = "data/filtered-referenced-eeg"
target_dir = "data/ica-excluded-eeg"
os.makedirs(target_dir, exist_ok=True)

participants = [32, 33, 34, 35, 36, 38, 39, 40, 41, 42, 43, 44, 45,
46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62,
63, 64, 65]
ica_components = []

for participant in participants:
    cached_file = f"{cache_dir}/sub-0{participant}-task-eeg_filtered-
referenced.fif"
    target_file = f"{target_dir}/sub-0{participant}-task-eeg_ica-
cleaned.fif"
    if os.path.exists(target_file):
        print(f"ICA cleaned file exists for {participant} -
{target_file}. Skipping...")
        continue
    if os.path.exists(cached_file):
        try:
            print(f"Found file {cached_file}")
            raw = mne.io.read_raw_fif(cached_file, preload=True)
            ica = mne.preprocessing.ICA(n_components=0.99,
method="fastica", random_state=42)
            ica.fit(raw)
            ica_components.append(ica.n_components)
            ica_labels = label_components(raw, ica, "iclabel")
            excluded_components = []
            for idx, label in enumerate(ica_labels["labels"]):
                if label != "brain":
                    excluded_components.append(idx)
                    print(f"Excluding index {idx} from {participant}
because label is {label}")
            print(f"For participant {participant}, we are excluding
components {excluded_components}")
            ica.exclude = excluded_components
            eeg_ica_cleaned = ica.apply(raw.copy())
            eeg_ica_cleaned.save(target_file, overwrite=True)
            print(f"@ Saved file {target_file}")

```

```

        except Exception as error:
            print(f"Error {error} while trying to apply ICA to
participant {participant}")
        else:
            print(f"File - {cached_file} not found!")

```

Epoching the data

```

## LABEL = the event string like x500913
## CODE = a numeric code of the event

def parse_label(label):
    if not label.startswith(("5", "6")) or len(label) > 7 or
len(label) < 6:
        print(f"Skipping this label: label {label} is invalid.")
        return None
    condition_flag = label[0]
    if condition_flag not in ("5", "6"):
        print(f"Error: label {label}. Condition flag is invalid")
    condition = "Memory" if condition_flag == "6" else "Listen"
    position = int(label[2:4])
    load = int(label[4:6])
    correct = None
    if condition == "Memory" and len(label) > 6:
        correct = "Correct" if label[6] == "1" else "Incorrect"
    return {"label" : label, "condition" : condition, "position" :
position, "load" : load, "correct" : correct}

def compute_power(epochs):
    freq = np.concatenate([np.arange(8, 13, 2), np.arange(30, 45, 2)])
    # Alpha and Gamma. We are using 45 because we cutoff after that in our
filter. Sparser sampling.
    n_cycles = freq / 2
    powers = epochs.compute_tfr(method="morlet", freqs=freq,
n_cycles=n_cycles, decim=4, average=False)
    powers.data = powers.data.astype(np.float32)
    # Compute power relative to a time window
    powers.apply_baseline(baseline=(-1.5, -0.2), mode="logratio")

    # Crop to get Alpha (8-12) and Gamma (30-44). Average across time,
frequencies, channels
    # This will give me a single value for whole scalp
    # FUTURE TODO: average over regions to get ROI specific values
    alpha_mean = powers.copy().crop(fmin=8,
fmax=12).data.mean(axis=(1, 2, 3))
    gamma_mean = powers.copy().crop(fmin=30,
fmax=44).data.mean(axis=(1, 2, 3))

    return pd.DataFrame({"alpha_power" : alpha_mean, "gamma_power" :

```

```

gamma_mean}))

preprocessed_eeg_dir = "data/ica-excluded-eeg"
def process_participant_data(num):
    print(f"Processing epoch for participant {num}")
    eeg_file = f"{preprocessed_eeg_dir}/sub-0{num}-task-eeg_ica-
cleaned.fif"
    if not os.path.exists(eeg_file):
        print(f"File {eeg_file} not found")
    else:
        raw = mne.io.read_raw_fif(eeg_file, preload=True)
        print(f"Read file {eeg_file}")
        events, event_id = mne.events_from_annotations(raw)
        code_to_label = {v : k for k, v in event_id.items()} # Reverse
map. Get a dict where event "code" is KEY and "label" is VALUE

        # Loop through events in time series
        metadata, valid_event_indices = [], []
        for index, event in enumerate(events):
            event_description = parse_label(code_to_label[event[2]]) # Get
the label from the event code which is in the 3rd position in "event"
            if event_description:
                metadata.append(event_description)
                valid_event_indices.append(index)

        metadata_df = pd.DataFrame(metadata) # Create metadata dataframe
        valid_events = events[valid_event_indices] # Get the list of clean
event codes
        matched_event_labels = {k: v for k, v in event_id.items() if
parse_label(k)} # Get the dictionary of event labels to codes, but
only where it's valid label

        # Create epoch object: from 1.5 before to 3.5 as authors did
        epochs = mne.Epochs(raw, events=valid_events,
event_id=matched_event_labels, tmin=-1.5, tmax=3.5,
metadata=metadata_df)

        print(f"Start computing power for participant {num}")
        powers = compute_power(epochs)
        participant_df = epochs.metadata.reset_index(drop=True).copy()
        participant_df["alpha_power"] = powers["alpha_power"].values
        participant_df["gamma_power"] = powers["gamma_power"].values
        participant_df["participant"] = num
        print(f"\nBelow is dataframe for participant {num}")
        display(participant_df)
        return participant_df

participants_batch_1 = [32, 33, 34, 35, 36, 38, 39, 40, 41, 42]
participants_batch_2 = [43, 44, 45, 46, 47, 48, 49, 50, 51, 52]

```

```

participants_batch_3 = [53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63,
64, 65] # We will leave this out for lack of time

all_participants = []

print(f"Starting batch_1. all_participants shape:
{len(all_participants)}")
for participant in participants_batch_1:
    print(f"\n\nProcessing epoch and power for participant
{participant}")
    # try:
    all_participants.append(process_participant_data(participant))
    # except Exception as error:
    #     print(f"Skipped participant {participant}. Error {error}.")

print(f"Starting batch_2. all_participants shape:
{len(all_participants)}")
for participant in participants_batch_2:
    print(f"\n\nProcessing epoch and power for participant
{participant}")
    try:
        all_participants.append(process_participant_data(participant))
    except Exception as error:
        print(f"Skipped participant {participant}. Error {error}.")

all_participants_df = pd.concat(all_participants, ignore_index=True)
display(all_participants)

print(f"all_participants shape is {len(all_participants)}")
display(all_participants[0])
print(f"all_participants_df shape os {all_participants_df.shape}")
all_participants_df.to_csv("eeg-power-features.csv", index=False)

```

Test cases

```

## Test the parse function: IT WORKS! :)
# labels = ["x500813", "x500913", "x501013", "x501113", "x501213",
"x501313", "x6001050", "x6001051", "x6002050", "x6002051", "x6003050",
"x6003051", "x6004050", "x6004051", "x6005050", "x6005051",
"x6001090", "x6001091", "x6002090"]
# for label in labels:
#     print(parse_label(label))

## Testing `mne.events_from_annotations`
# test_raw = mne.io.read_raw_fif("data/ica-excluded-eeg/sub-051-task-
eeg_ica-cleaned.fif", preload=True)
# events, event_id = mne.events_from_annotations(test_raw)
# print(f"Events : {events}")
# print(f"Event ID: {event_id}")

```

EEG pre-processing notes

I filtered the EEG data by applying 1Hz high-pass and 45Hz low-pass filters. This was sufficient to handle low-frequency drift and power line noise. I also re-referenced each channel reading to the average value. Both these steps were in accordance with what the original authors did in their experimental paper. I removed artifacts automatically using `mne-icalabel` which classifies individual components into one of seven categories: Brain, Muscle, Eye, Heart, Line Noise, Channel Noise, Other. I removed every component that was not categorized as **Brain**.

I created epochs around the event code, to extract time windows around specific events during the test. Then I computed whole brain alpha and gamma power in that specific epoch.

Kosachenko, A. I., Kasanov, D., Kotyusov, A. I., & Pavlov, Y. G. (2023). EEG and pupillometric signatures of working memory overload. *Psychophysiology*, 60(6), e14275.
<https://doi.org/10.1111/psyp.14275>

Classify: Determine condition (memory -vs- listen) from whole brain alpha and gamma

Load data

```
df_features = pd.read_csv("data/eeg-power-features.csv")
print(f"DF: {df_features.shape}")
display(df_features)
```

DF: (29187, 8)

```
{
  "columns": [
    {"name": "index", "rawType": "int64", "type": "integer"},
    {"name": "label", "rawType": "int64", "type": "integer"},
    {"name": "condition", "rawType": "str", "type": "string"},
    {"name": "position", "rawType": "int64", "type": "integer"},
    {"name": "load", "rawType": "int64", "type": "integer"},
    {"name": "correct", "rawType": "str", "type": "string"},
    {"name": "alpha_power", "rawType": "float64", "type": "float"},
    {"name": "gamma_power", "rawType": "float64", "type": "float"},
    {"name": "participant", "rawType": "int64", "type": "integer"}
  ],
  "ref": "d9d23d2b-e882-41dd-a3fd-fbadad3ac1b0",
  "rows": [
    [
      "0", "500113", "Listen", "1", "13", null, "-0.05747722", "-0.19253373", "32"
    ],
    [
      "1", "500213", "Listen", "2", "13", null, "-0.39439785", "-0.184071", "32"
    ],
    [
      "2", "500313", "Listen", "3", "13", null, "-0.41292822", "-0.2058728", "32"
    ],
    [
      "3", "500413", "Listen", "4", "13", null, "-0.06700084", "-0.22936772", "32"
    ],
    [
      "4", "500513", "Listen", "5", "13", null, "-0.25380316", "-0.2323368", "32"
    ],
    [
      "5", "500613", "Listen", "6", "13", null, "-0.22311088", "-0.20582938", "32"
    ],
    [
      "6", "500713", "Listen", "7", "13", null, "-0.3041852", "-0.23092456", "32"
    ],
    [
      "7", "500813", "Listen", "8", "13", null, "-0.38281012", "-0.21458986", "32"
    ],
    [
      "8", "500913", "Listen", "9", "13", null, "-0.06594014", "-0.21206291", "32"
    ],
    [
      "9", "501013", "Listen", "10", "13", null, "-0.20998463", "-0.33647376", "32"
    ]
  ]
}
```

```
[ "10", "501113", "Listen", "11", "13", null, "-0.07668598", "-  
0.26485136", "32"], [ "11", "501213", "Listen", "12", "13", null, "-  
0.7411584", "-0.24243137", "32"],  
[ "12", "501313", "Listen", "13", "13", null, "-0.36447027", "-  
0.2080821", "32"], [ "13", "500105", "Listen", "1", "5", null, "-0.29906842", "-  
0.50444335", "32"], [ "14", "500205", "Listen", "2", "5", null, "-  
0.34630018", "-0.28405112", "32"],  
[ "15", "500305", "Listen", "3", "5", null, "-0.085187264", "-  
0.09976511", "32"], [ "16", "500405", "Listen", "4", "5", null, "-  
0.24121489", "-0.30834353", "32"],  
[ "17", "500505", "Listen", "5", "5", null, "-0.03760033", "-  
0.23441376", "32"], [ "18", "500109", "Listen", "1", "9", null, "-  
0.16558225", "-0.175408", "32"], [ "19", "500209", "Listen", "2", "9", null, "-  
0.32311538", "-0.24963427", "32"],  
[ "20", "500309", "Listen", "3", "9", null, "-0.034109086", "-  
0.28247163", "32"], [ "21", "500409", "Listen", "4", "9", null, "-  
0.08446263", "-0.17050813", "32"],  
[ "22", "500509", "Listen", "5", "9", null, "-0.5458342", "-  
0.121628754", "32"], [ "23", "500609", "Listen", "6", "9", null, "-  
0.19654635", "-0.24728476", "32"],  
[ "24", "500709", "Listen", "7", "9", null, "-0.12493693", "-  
0.24478137", "32"], [ "25", "500809", "Listen", "8", "9", null, "-  
0.21585643", "-0.2386734", "32"], [ "26", "500909", "Listen", "9", "9", null, "-  
0.0057429173", "-0.21005195", "32"],  
[ "27", "6001051", "Memory", "1", "5", "Correct", "0.043831125", "-  
0.15316334", "32"], [ "28", "6002051", "Memory", "2", "5", "Correct", "-  
0.09416469", "-0.24690044", "32"],  
[ "29", "6003051", "Memory", "3", "5", "Correct", "-0.30560228", "-  
0.26021716", "32"], [ "30", "6004051", "Memory", "4", "5", "Correct", "-  
0.22176857", "-0.24329819", "32"],  
[ "31", "6005051", "Memory", "5", "5", "Correct", "-0.43997502", "-  
0.21715133", "32"], [ "32", "6001091", "Memory", "1", "9", "Correct", "-  
0.3922449", "-0.22304542", "32"],  
[ "33", "6002091", "Memory", "2", "9", "Correct", "-0.26948857", "-  
0.3158688", "32"], [ "34", "6003091", "Memory", "3", "9", "Correct", "-  
0.5117146", "-0.1462156", "32"],  
[ "35", "6004090", "Memory", "4", "9", "Incorrect", "-0.23835337", "-  
0.2155481", "32"], [ "36", "6005090", "Memory", "5", "9", "Incorrect", "-  
0.02668732", "-0.24127243", "32"],  
[ "37", "6006090", "Memory", "6", "9", "Incorrect", "-0.25725392", "-  
0.26035413", "32"], [ "38", "6007090", "Memory", "7", "9", "Incorrect", "-  
0.0034819392", "-0.25389126", "32"],  
[ "39", "6008090", "Memory", "8", "9", "Incorrect", "-0.249232", "-  
0.17769654", "32"], [ "40", "6009090", "Memory", "9", "9", "Incorrect", "-  
0.45456257", "-0.2186912", "32"],  
[ "41", "6001130", "Memory", "1", "13", "Incorrect", "-0.408934", "-  
0.239864", "32"], [ "42", "6002130", "Memory", "2", "13", "Incorrect", "-  
0.27295393", "-0.19897458", "32"],  
[ "43", "6003130", "Memory", "3", "13", "Incorrect", "-0.22254254", "-
```



```

0.23279098", "32"], ["44", "6004130", "Memory", "4", "13", "Incorrect", "-
0.20219196", "-0.20317806", "32"],
["45", "6005130", "Memory", "5", "13", "Incorrect", "-0.17790934", "-
0.24266884", "32"], ["46", "6006130", "Memory", "6", "13", "Incorrect", "-
0.32935804", "-0.20035398", "32"],
["47", "6007130", "Memory", "7", "13", "Incorrect", "-0.2268287", "-
0.22601052", "32"], ["48", "6008130", "Memory", "8", "13", "Incorrect", "-
0.098484516", "-0.26363048", "32"],
["49", "6009130", "Memory", "9", "13", "Incorrect", "-0.21008226", "-
0.16530764", "32"]], "shape": {"columns": 8, "rows": 29187}}

X = df_features[["alpha_power", "gamma_power"]]
Y = df_features["condition"]
group = df_features["participant"]
display(Y.value_counts())

{"columns": [{"name": "condition", "rawType": "str", "type": "string"},
{"name": "count", "rawType": "int64", "type": "integer"}], "ref": "ac0efa69-
e162-4ac2-8125-0a90db1169a2", "rows": [{"Memory", "19440"},
["Listen", "9747"]], "shape": {"columns": 1, "rows": 2}}

```

My dataset has 19440 memory events and 9747 listen events. To manage this, I will run the models with `class_weight=balanced` but this still might cause problems.

Hyperparameter tuning using RandomizedSearchCV

```

logo = LeaveOneGroupOut()

param_grid = {
    "n_estimators" : [50, 100, 200, 300, 400],
    "max_depth" : [10, 15, 20, 25, 30],
    "min_samples_split" : [2, 5, 10, 15, 20, 25]
}

random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42, class_weight="balanced"),
    param_distributions = param_grid,
    n_iter = 25,
    cv = list(logo.split(X, Y, groups=group)),
    scoring = "f1_macro",
    random_state = 42,
    n_jobs = -1
)

random_search.fit(X, Y)
print(f"The best params: {random_search.best_params_}")
print(f"The best score: {random_search.best_score_}")

```

```
The best params: {'n_estimators': 100, 'min_samples_split': 25,
'max_depth': 10}
The best score: 0.5108834259391384
```

Hyperparameter tuning conclusion

With n=25 run, I got:

- The best params: {'n_estimators': 100, 'min_samples_split': 25, 'max_depth': 10}
- The best f1_macro score: 0.5108834259391384

I will use this best_params in running my random forest model.

Running my random forest

```
scores_df = pd.DataFrame(columns=["model", "metric", "score"])

best_params = random_search.best_params_
accuracies, true, predicted = [], [], []
accuracy

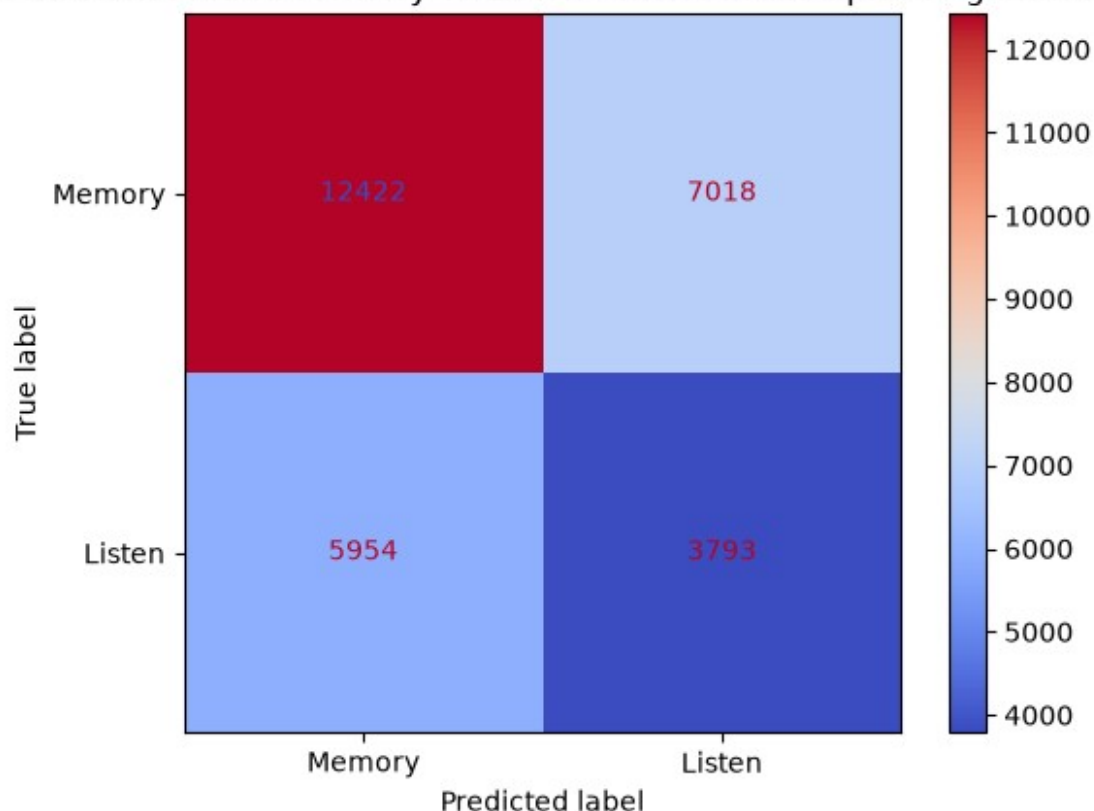
for train, test in logo.split(X, Y, groups=group):
    classifier = RandomForestClassifier(**best_params,
random_state=42, class_weight="balanced")
    classifier.fit(X.iloc[train], Y.iloc[train])
    prediction = classifier.predict(X.iloc[test])
    accuracy = accuracy_score(Y.iloc[test], prediction)
    accuracies.append(accuracy)
    true.extend(Y.iloc[test])
    predicted.extend(prediction)

cm = confusion_matrix(true, predicted, labels=["Memory", "Listen"])
ConfusionMatrixDisplay(cm, display_labels=["Memory",
"Listen"]).plot(cmap="coolwarm")
plt.title("Confusion Matrix: Memory vs Listen - whole brain alpha vs
gamma")
plt.show()

report = classification_report(true, predicted, output_dict = True)
report_df = pd.DataFrame(report).transpose().round(3)

scores_df.loc[len(scores_df)] = ["our-model", "accuracy", accuracy]
scores_df.loc[len(scores_df)] = ["our-model", "f1-macro",
report_df.loc["macro avg", "f1-score"]]
scores_df.loc[len(scores_df)] = ["our-model", "f1-avg",
report_df.loc["weighted avg", "f1-score"]]
display(report_df)
display(scores_df)
```

Confusion Matrix: Memory vs Listen - whole brain alpha vs gamma



```
{
  "columns": [
    { "name": "index", "rawType": "str", "type": "string" },
    { "name": "precision", "rawType": "float64", "type": "float" },
    { "name": "recall", "rawType": "float64", "type": "float" },
    { "name": "f1-score", "rawType": "float64", "type": "float" },
    { "name": "support", "rawType": "float64", "type": "float" }
  ],
  "ref": "d37dd1e2-0ddd-4a36-87b6-59da5a4413f3",
  "rows": [
    [ "Listen", "0.351", "0.389", "0.369", "9747.0" ],
    [ "Memory", "0.676", "0.639", "0.657", "19440.0" ],
    [ "accuracy", "0.556", "0.556", "0.556", "0.556" ],
    [ "macro avg", "0.513", "0.514", "0.513", "29187.0" ],
    [ "weighted avg", "0.567", "0.556", "0.561", "29187.0" ]
  ],
  "shape": { "columns": 4, "rows": 5 }
}
```

```
{
  "columns": [
    { "name": "index", "rawType": "int64", "type": "integer" },
    { "name": "model", "rawType": "str", "type": "string" },
    { "name": "metric", "rawType": "str", "type": "string" },
    { "name": "score", "rawType": "float64", "type": "float" }
  ],
  "ref": "00fefa07-2d2d-4f00-9977-e0f981f607a8",
  "rows": [
    [ "0", "our-model", "accuracy", "0.5727023319615913" ],
    [ "1", "our-model", "f1-macro", "0.513" ],
    [ "2", "our-model", "f1-avg", "0.561" ]
  ],
  "shape": { "columns": 3, "rows": 3 }
}
```

Comparing with DummyClassifier as a baseline

```
logo = LeaveOneGroupOut()
dummy_true, dummy_pred, accuracy = [], [], []
accuracy
for train, test in logo.split(X, Y, groups=group):
    dummy = DummyClassifier(strategy='most_frequent')
    dummy.fit(X.iloc[train], Y.iloc[train])
    preds = dummy.predict(X.iloc[test])
    accuracy = accuracy_score(Y.iloc[test], preds)
    accuracies.append(accuracy)
    dummy_true.extend(Y.iloc[test])
    dummy_pred.extend(preds)

report = classification_report(dummy_true, dummy_pred, output_dict =
True)
report_df = pd.DataFrame(report).transpose().round(3)
display(report_df)
scores_df.loc[len(scores_df)] = ["dummy-most-frequent", "accuracy",
accuracy]
scores_df.loc[len(scores_df)] = ["dummy-most-frequent", "f1-macro",
report_df.loc["macro avg", "f1-score"]]
scores_df.loc[len(scores_df)] = ["dummy-most-frequent", "f1-avg",
report_df.loc["weighted avg", "f1-score"]]
display(scores_df)

c:\Users\atifm\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sklearn\metrics\_classification.py:1879:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
in labels with no predicted samples. Use `zero_division` parameter to
control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])
c:\Users\atifm\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sklearn\metrics\_classification.py:1879:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
in labels with no predicted samples. Use `zero_division` parameter to
control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])
c:\Users\atifm\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sklearn\metrics\_classification.py:1879:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
in labels with no predicted samples. Use `zero_division` parameter to
control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])

{"columns":[{"name":"index","rawType":"str","type":"string"},
{"name":"precision","rawType":"float64","type":"float"}],
```

```
{
  "name": "recall", "rawType": "float64", "type": "float"},
  {"name": "f1-score", "rawType": "float64", "type": "float"},
  {"name": "support", "rawType": "float64", "type": "float"}],
  "ref": "51cc6186-df27-48f6-baaa-bb4cbcafe18b",
  "rows": [
    ["Listen", "0.0", "0.0", "0.0", "9747.0"],
    ["Memory", "0.666", "1.0", "0.8", "19440.0"],
    ["accuracy", "0.666", "0.666", "0.666", "0.666"],
    ["macro avg", "0.333", "0.5", "0.4", "29187.0"],
    ["weighted avg", "0.444", "0.666", "0.533", "29187.0"]],
  "shape": {"columns": 4, "rows": 5}}

```

```
{
  "columns": [
    {"name": "index", "rawType": "int64", "type": "integer"},
    {"name": "model", "rawType": "str", "type": "string"},
    {"name": "metric", "rawType": "str", "type": "string"},
    {"name": "score", "rawType": "float64", "type": "float"}],
  "ref": "21f66e42-59fd-467f-a24e-b7fe250dfa34",
  "rows": [
    ["0", "our-model", "accuracy", "0.5727023319615913"],
    ["1", "our-model", "f1-macro", "0.513"],
    ["2", "our-model", "f1-avg", "0.561"],
    ["3", "dummy-most-frequent", "accuracy", "0.6666666666666666"],
    ["4", "dummy-most-frequent", "f1-macro", "0.4"],
    ["5", "dummy-most-frequent", "f1-avg", "0.533"]],
  "shape": {"columns": 3, "rows": 6}}

```

```
logo = LeaveOneGroupOut()
dummy_true, dummy_pred = [], []

```

```
for train, test in logo.split(X, Y, groups=group):
    dummy = DummyClassifier(strategy='stratified')
    dummy.fit(X.iloc[train], Y.iloc[train])
    preds = dummy.predict(X.iloc[test])

```

```
    dummy_true.extend(Y.iloc[test])
    dummy_pred.extend(preds)

```

```
report = classification_report(dummy_true, dummy_pred, output_dict =
True)

```

```
report_df = pd.DataFrame(report).transpose().round(3)

```

```
display(report_df)

```

```
scores_df.loc[len(scores_df)] = ["dummy-stratified", "accuracy",
accuracy]

```

```
scores_df.loc[len(scores_df)] = ["dummy-stratified", "f1-macro",
report_df.loc["macro avg", "f1-score"]]

```

```
scores_df.loc[len(scores_df)] = ["dummy-stratified", "f1-avg",
report_df.loc["weighted avg", "f1-score"]]

```

```
display(scores_df)

```

```
{
  "columns": [
    {"name": "index", "rawType": "str", "type": "string"},
    {"name": "precision", "rawType": "float64", "type": "float"},
    {"name": "recall", "rawType": "float64", "type": "float"},
    {"name": "f1-score", "rawType": "float64", "type": "float"},
    {"name": "support", "rawType": "float64", "type": "float"}],
  "ref": "08e7b21e

```

```

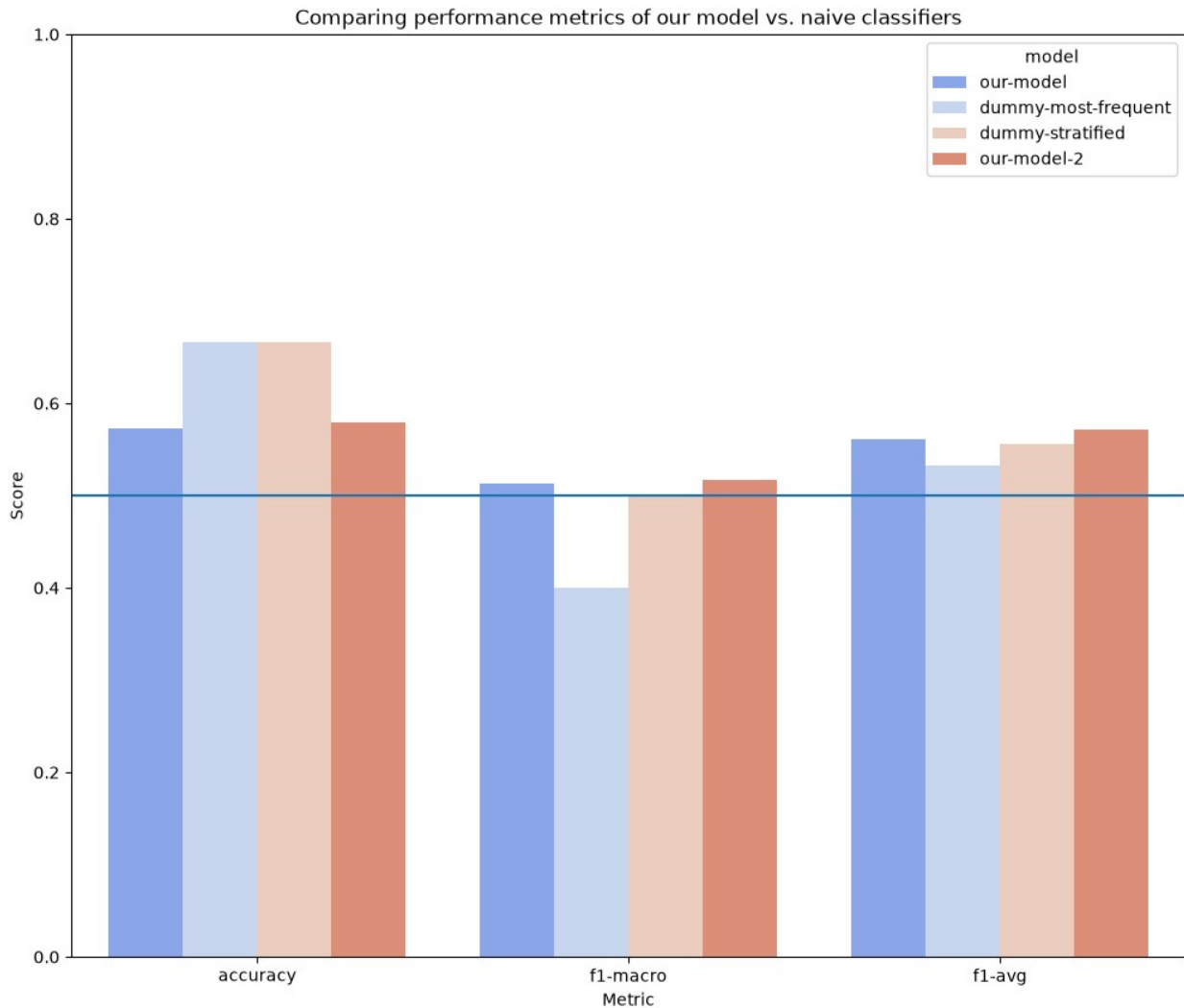
-7ee8-49fd-8bf5-d086088ff530", "rows":
[["Listen", "0.335", "0.335", "0.335", "9747.0"],
["Memory", "0.667", "0.667", "0.667", "19440.0"],
["accuracy", "0.556", "0.556", "0.556", "0.556"], ["macro
avg", "0.501", "0.501", "0.501", "29187.0"], ["weighted
avg", "0.556", "0.556", "0.556", "29187.0"]], "shape":
{"columns":4, "rows":5}}

{"columns":[{"name":"index", "rawType":"int64", "type":"integer"},
{"name":"model", "rawType":"str", "type":"string"},
{"name":"metric", "rawType":"str", "type":"string"},
{"name":"score", "rawType":"float64", "type":"float"}], "ref":"680f9685-
2e50-45a2-a133-7dffafcc8e30", "rows":[["0", "our-
model", "accuracy", "0.5727023319615913"], ["1", "our-model", "f1-
macro", "0.513"], ["2", "our-model", "f1-avg", "0.561"], ["3", "dummy-most-
frequent", "accuracy", "0.6666666666666666"], ["4", "dummy-most-
frequent", "f1-macro", "0.4"], ["5", "dummy-most-frequent", "f1-
avg", "0.533"], ["6", "dummy-
stratified", "accuracy", "0.6666666666666666"], ["7", "dummy-
stratified", "f1-macro", "0.501"], ["8", "dummy-stratified", "f1-
avg", "0.556"]], "shape":{"columns":3, "rows":9}}

plt.figure(figsize=(12, 10))
sns.barplot(x="metric", y="score", hue="model", data=scores_df,
palette="coolwarm")
plt.title("Comparing performance metrics of our model vs. naive
classifiers")
plt.xlabel("Metric")
plt.ylabel("Score")
plt.axhline(0.5)
plt.ylim(0, 1)

(0.0, 1.0)

```



Markdown conclusion

Running my model, I get the following scores. Mean accuracy = 0.573, f1_macro = 0.513, and f1_weighted_average = 0.561. When it comes to class-specific metrics, my model can predict the memory condition with a precision of 0.676, a recall of 0.639, and an f1 of 0.657.

At first glance, this might look extremely promising, but note the class imbalance. We have roughly 2:1 ratio of memory to listen trials, so even with `class_weight = balanced`, we need to be careful before accepting this conclusion.

To set a quantifiable baseline, we will run a `DummyClassifier` to see if our model is performing better than a naive estimation. With a dummy classifier that always picks the most frequent, i.e. memory, we get a mean accuracy of 0.666, f1_macro = 0.4, and f1_weighted_average = 0.53. The accuracy of the dummy classifier is pulled up by the sheer proportion of the memory trials.

Analysing another dummy classifier with a "stratified" strategy, where it guesses in proportion of the frequency of labels, it has a f1_macro of 0.50 and an f1_weighted_average of 0.56.

Concluding this section, my model performs slightly better than naive classifier. However, it is likely that we can improve our model and the following sections will work on that.

Below, I have a section to diagnose why my model is not performing better. As we see, between conditions, the whole brain alpha and gamma don't vary much.

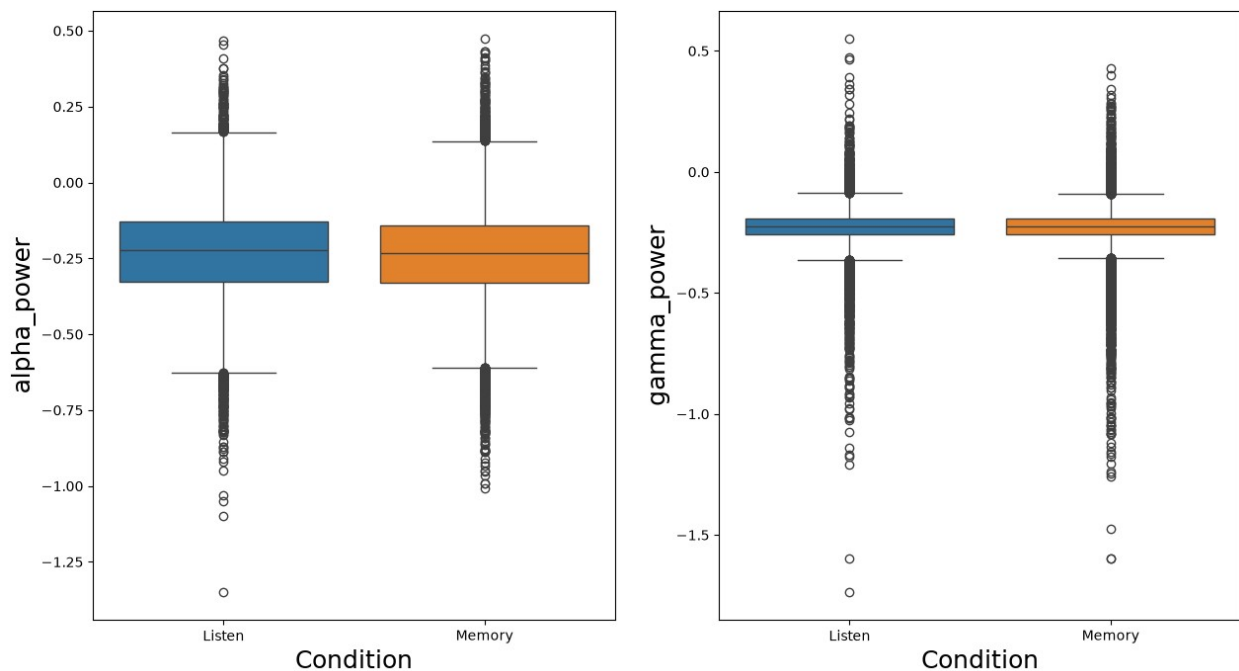
Diagnosing model performance

```
waves = ["alpha", "gamma"]
fig, ax = plt.subplots(1, 2, figsize=(15,8))

for column, wave in enumerate(waves):
    column_name = f"{wave}_power"
    sns.boxplot(x="condition", y=column_name, ax=ax[column],
data=df_features, hue="condition")
    ax[column].set_ylabel(column_name, fontsize=18)
    ax[column].set_xlabel("Condition", fontsize=18)

plt.suptitle("Differences in alpha and gamma power by condition",
fontsize=20)
plt.show()
```

Differences in alpha and gamma power by condition



Classify: Determine condition (memory -vs- listen) from *ROI specific* alpha and gamma

We are going to redo our random forest, but this time with specific brain region powers instead of collapsing it to a global average. See below...

Compute powers

```
## LABEL = the event string like x500913
## CODE = a numeric code of the event

### SAME AS RANDOM FOREST 1
def parse_label(label):
    if not label.startswith(("5", "6")) or len(label) > 7 or
len(label) < 6:
        print(f"Skipping this label: label {label} is invalid.")
        return None
    condition_flag = label[0]
    if condition_flag not in ("5", "6"):
        print(f"Error: label {label}. Condition flag is invalid")
    condition = "Memory" if condition_flag == "6" else "Listen"
    position = int(label[2:4])
    load = int(label[4:6])
    correct = None
    if condition == "Memory" and len(label) > 6:
        correct = "Correct" if label[6] == "1" else "Incorrect"
    return {"label" : label, "condition" : condition, "position" :
position, "load" : load, "correct" : correct}

#####
#####
# CHANNELS ARE: The channel names are: ['Fp1', 'Fz', 'F3', 'F7',
'FT9',
# 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz', 'P3', 'P7',
# 'O1', 'Oz', 'O2', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz', 'C4',
# 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3', 'AFz',
# 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07',
# 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8', 'C6',
'C2',
# 'FC4', 'FT8', 'F6', 'AF8', 'AF4', 'F2']

# Groups defined as per: https://neuroanalyzer.org/tutorials/locs.html
roi_groups = {
    "frontal" : ["Fp1", "Fp2", "F3", "F4", "F7", "F8", "Fz"], #
Frontal lobe: Executive function, attention, decision-making, and
working memory.
    "parietal" : ["P3", "P4", "P7", "P8", "Pz", "CP1", "CP2", "CP5",
"CP6"], # Parietal lobe: Spatial orientation, sensory integration, and
attention.
    "temporal": ["T7", "T8", "F7", "F8", "FT9", "FT10"], # Temporal
lobe: Auditory processing, language, and memory.
    "midline" : ["Fz", "Cz", "Pz", "Oz"], # Midline structures:
Default mode network, global brain states.
}
```

```

# Thus we will have:
# frontal_alpha, frontal_gamma
# parietal_alpha, parietal_gamma
# temporal_alpha, temporal_gamma
# midline_alpha, midline_gamma

def compute_power(epochs):
    freq = np.concatenate([np.arange(8, 13, 2), np.arange(30, 45, 2)])
    # Alpha and Gamma. We are using 45 because we cutoff after that in our
    # filter. Sparser sampling.
    n_cycles = freq / 2
    powers = epochs.compute_tfr(method="morlet", freqs=freq,
n_cycles=n_cycles, decim=4, average=False)
    powers.data = powers.data.astype(np.float32)
    # Compute power relative to a time window
    powers.apply_baseline(baseline=(-1.5, -0.2), mode="logratio")

    alpha = powers.copy().crop(fmin=8, fmax=12)
    gamma = powers.copy().crop(fmin=30, fmax=44)

    # The key difference here: get ROI specific channels
    # I can do list construction one-line but I like this more
    ch_indices = []
    roi_powers = {}
    for roi_name, ch_list in roi_groups.items():
        ch_indices = []
        for ch in ch_list:
            if ch in powers.ch_names:
                ch_indices.append(powers.ch_names.index(ch))
        roi_powers[f"{roi_name}_alpha"] = alpha.data[:, ch_indices, :,
:].mean(axis=(1, 2, 3))
        roi_powers[f"{roi_name}_gamma"] = gamma.data[:, ch_indices, :,
:].mean(axis=(1, 2, 3))

    return pd.DataFrame(roi_powers)
#####

### SAME AS RANDOM FOREST 1
preprocessed_eeg_dir = "data/ica-excluded-eeg"
def process_participant_data(num):
    print(f"Processing epoch for participant {num}")
    eeg_file = f"{preprocessed_eeg_dir}/sub-0{num}-task-eeg_ica-
cleaned.fif"
    if not os.path.exists(eeg_file):
        print(f"File {eeg_file} not found")
    else:
        raw = mne.io.read_raw_fif(eeg_file, preload=True)
        print(f"Read file {eeg_file}")
        events, event_id = mne.events_from_annotations(raw)

```

```

        code_to_label = {v : k for k, v in event_id.items()} # Reverse
map. Get a dict where event "code" is KEY and "label" is VALUE

    # Loop through events in time series
    metadata, valid_event_indices = [], []
    for index, event in enumerate(events):
        event_description = parse_label(code_to_label[event[2]]) # Get
the label from the event code which is in the 3rd position in "event"
        if event_description:
            metadata.append(event_description)
            valid_event_indices.append(index)

    metadata_df = pd.DataFrame(metadata) # Create metadata dataframe
    valid_events = events[valid_event_indices] # Get the list of clean
event codes
    matched_event_labels = {k: v for k, v in event_id.items() if
parse_label(k)} # Get the dictionary of event labels to codes, but
only where it's valid label

    # Create epoch object: from 1.5 before to 3.5 as authors did
    epochs = mne.Epochs(raw, events=valid_events,
event_id=matched_event_labels, tmin=-1.5, tmax=3.5,
metadata=metadata_df)

    print(f"Start computing power for participant {num}")

    powers = compute_power(epochs)
    participant_df = epochs.metadata.reset_index(drop=True).copy()

    # Thus we will have:
    # frontal_alpha, frontal_gamma
    # parietal_alpha, parietal_gamma
    # temporal_alpha, temporal_gamma
    # midline_alpha, midline_gamma

    participant_df["frontal_alpha"] = powers["frontal_alpha"].values
    participant_df["frontal_gamma"] = powers["frontal_gamma"].values

    participant_df["parietal_alpha"] = powers["parietal_alpha"].values
    participant_df["parietal_gamma"] = powers["parietal_gamma"].values

    participant_df["temporal_alpha"] = powers["temporal_alpha"].values
    participant_df["temporal_gamma"] = powers["temporal_gamma"].values

    participant_df["midline_alpha"] = powers["midline_alpha"].values
    participant_df["midline_gamma"] = powers["midline_gamma"].values

    participant_df["participant"] = num

    print(f"\nBelow is dataframe for participant {num}")

```

```

        display(participant_df)
        return participant_df

participants_batch_1 = [32, 33, 34, 35, 36, 38, 39, 40, 41, 42]
participants_batch_2 = [43, 44, 45, 46, 47, 48, 49, 50, 51, 52]
participants_batch_3 = [53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63,
64, 65] # We will leave this out for lack of time

all_participants_2 = []

print(f"Starting batch_1. all_participants shape:
{len(all_participants_2)}")
for participant in participants_batch_1:
    print(f"\n\nProcessing epoch and power for participant
{participant}")
    # try:
    all_participants_2.append(process_participant_data(participant))
    # except Exception as error:
    #     print(f"Skipped participant {participant}. Error {error}.")

print(f"Starting batch_2. all_participants shape:
{len(all_participants_2)}")
for participant in participants_batch_2:
    print(f"\n\nProcessing epoch and power for participant
{participant}")
    try:

all_participants_2.append(process_participant_data(participant))
    except Exception as error:
        print(f"Skipped participant {participant}. Error {error}.")

all_participants_df_2 = pd.concat(all_participants_2,
ignore_index=True)
display(all_participants_df_2)
all_participants_df_2.to_csv("data/eeg-power-features-per-roi.csv",
index=False)

```

Load Data

```

df_features_2 = pd.read_csv("data/eeg-power-features-per-roi.csv")
print(f"DF: {df_features_2.shape}")
display(df_features_2)

```

DF: (29187, 14)

```

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"label","rawType":"int64","type":"integer"},
{"name":"condition","rawType":"str","type":"string"},
{"name":"position","rawType":"int64","type":"integer"},
{"name":"load","rawType":"int64","type":"integer"},
{"name":"correct","rawType":"str","type":"string"},

```

```
{
  "name": "frontal_alpha", "rawType": "float64", "type": "float"},
  "name": "frontal_gamma", "rawType": "float64", "type": "float"},
  "name": "parietal_alpha", "rawType": "float64", "type": "float"},
  "name": "parietal_gamma", "rawType": "float64", "type": "float"},
  "name": "temporal_alpha", "rawType": "float64", "type": "float"},
  "name": "temporal_gamma", "rawType": "float64", "type": "float"},
  "name": "midline_alpha", "rawType": "float64", "type": "float"},
  "name": "midline_gamma", "rawType": "float64", "type": "float"},
  "name": "participant", "rawType": "int64", "type": "integer"}], "ref": "4eef55e6-4ff5-4dcc-b83e-76b7938a8c43", "rows":
[["0", "500113", "Listen", "1", "13", null, "-0.025957057", "-0.18326485", "-0.009031365", "-0.20584455", "-0.15735893", "-0.19449925", "-0.09644044", "-0.16075216", "32"],
["1", "500213", "Listen", "2", "13", null, "-0.4295599", "-0.1586829", "-0.39929757", "-0.2581144", "-0.36366075", "-0.1721333", "-0.40252006", "-0.16622323", "32"],
["2", "500313", "Listen", "3", "13", null, "-0.42290002", "-0.24490291", "-0.4255417", "-0.17459345", "-0.30750823", "-0.2257737", "-0.41035983", "-0.22974303", "32"],
["3", "500413", "Listen", "4", "13", null, "-0.06936548", "-0.21740115", "-0.060178876", "-0.20385209", "0.06379208", "-0.21597047", "-0.11895034", "-0.21043433", "32"],
["4", "500513", "Listen", "5", "13", null, "-0.30864483", "-0.2719169", "-0.2301973", "-0.2210616", "-0.36474884", "-0.2684594", "-0.2772169", "-0.25362664", "32"],
["5", "500613", "Listen", "6", "13", null, "-0.28857574", "-0.19293183", "-0.16426952", "-0.21656498", "-0.2809203", "-0.16254508", "-0.21992949", "-0.18849497", "32"],
["6", "500713", "Listen", "7", "13", null, "-0.20350565", "-0.21727931", "-0.32124722", "-0.24504736", "-0.1468042", "-0.22221668", "-0.30233803", "-0.27994496", "32"],
["7", "500813", "Listen", "8", "13", null, "-0.34081808", "-0.22551022", "-0.45709014", "-0.22024778", "-0.41842172", "-0.23807715", "-0.3539373", "-0.20218052", "32"],
["8", "500913", "Listen", "9", "13", null, "-0.025039962", "-0.18180633", "-0.082695514", "-0.22573274", "-0.06977223", "-0.27738345", "-0.080132455", "-0.09608986", "32"],
["9", "501013", "Listen", "10", "13", null, "-0.22259584", "-0.32267013", "-0.2140352", "-0.37525222", "-0.22824852", "-0.33167717", "-0.1656693", "-0.29273036", "32"],
["10", "501113", "Listen", "11", "13", null, "-0.09276553", "-0.2743676", "-0.049321573", "-0.20778574", "-0.089088164", "-0.2447739", "-0.10138233", "-0.32953507", "32"],
["11", "501213", "Listen", "12", "13", null, "-0.9575961", "-0.3040309", "-0.691751", "-0.22905163", "-0.6792006", "-0.2936776", "-0.7334833", "-0.26064476", "32"],
["12", "501313", "Listen", "13", "13", null, "-0.42778593", "-0.21990559", "-0.2722173", "-0.2260552", "-0.48790473", "-0.2311506", "-0.32097512", "-0.17965907", "32"],
["13", "500105", "Listen", "1", "5", null, "-0.27830294", "-0.6870689", "-0.23974974", "-0.42242566", "-0.44163892", "-0.7199085", "-0.39450857", "-0.37641114", "32"],
["14", "500205", "Listen", "2", "5", null, "-0.34897283", "-0.31278422", "-0.3084698", "-0.29483205", "-0.41784438", "-0.26977026", "-0.25131318", "-0.30931133", "32"],
["15", "500305", "Listen", "3", "5", null, "-0.2729121", "-0.06319353", "-
```

0.0009827339","-0.063633464","0.05674658","-0.1197799","-
0.07165969","-0.14079067","32"],
["16","500405","Listen","4","5",null,"-0.063734435","-0.27875915","-
0.33953163","-0.34841815","-0.33590022","-0.28846663","-0.19536875","-
0.29774624","32"],
["17","500505","Listen","5","5",null,"0.100504465","-0.29090923","-
0.092323095","-0.2601843","0.08237771","-0.29721344","-0.014801744","-
0.145563","32"],["18","500109","Listen","1","9",null,"-0.11886485","-
0.15496579","-0.14856766","-0.23000282","-0.21635072","-0.23587619","-
0.1684342","-0.11889206","32"],["19","500209","Listen","2","9",null,"-
0.33948734","-0.2840895","-0.35501134","-0.25531724","-0.51224536","-
0.2544201","-0.27235836","-0.27527538","32"],
["20","500309","Listen","3","9",null,"-0.012826612","-
0.30882943","0.022352863","-0.25369757","0.034773197","-0.27630496","-
0.13377067","-0.26401532","32"],
["21","500409","Listen","4","9",null,"-0.05910009","-0.17341319","-
0.094559416","-0.14685845","-0.12642705","-0.20388396","-
0.12183217","-0.19637032","32"],
["22","500509","Listen","5","9",null,"-0.5474208","-0.13229024","-
0.4890003","-0.09876043","-0.6448048","-0.1219852","-0.6356747","-
0.14665087","32"],["23","500609","Listen","6","9",null,"-0.2589021","-
0.21471645","-0.16241616","-0.23287363","-0.0867925","-0.2541058","-
0.2463156","-0.17246994","32"],["24","500709","Listen","7","9",null,"-
0.1014311","-0.22763164","-0.112267636","-0.23234826","-0.18075179","-
0.24761663","-0.12586412","-0.26313964","32"],
["25","500809","Listen","8","9",null,"-0.03089234","-0.2904603","-
0.3292658","-0.27815068","-0.1869417","-0.13189666","-0.09611091","-
0.26322964","32"],["26","500909","Listen","9","9",null,"-
0.055112306","-0.1931216","0.015092393","-0.2610998","0.120639585","-
0.15700145","-0.03282554","-0.20202817","32"],
["27","6001051","Memory","1","5","Correct","0.19306137","-
0.081946254","0.015768766","-0.157055","0.07461289","-
0.20597205","0.05658468","-0.12851547","32"],
["28","6002051","Memory","2","5","Correct","0.03840608","-
0.3173205","0.009145574","-0.19637242","-0.20130816","-0.3014595","-
0.20558067","-0.19098456","32"],
["29","6003051","Memory","3","5","Correct","-0.29856443","-
0.28146115","-0.29204983","-0.23172358","-0.31965402","-0.25834638","-
0.30473813","-0.33122146","32"],
["30","6004051","Memory","4","5","Correct","-0.2702053","-
0.24439706","-0.22737053","-0.27406314","-0.29719052","-0.22277696","-
0.21135351","-0.25489524","32"],
["31","6005051","Memory","5","5","Correct","-0.47628137","-
0.21425423","-0.4897416","-0.2394687","-0.3333102","-0.19943354","-
0.42209068","-0.18647537","32"],
["32","6001091","Memory","1","9","Correct","-0.35371333","-
0.21229637","-0.40571758","-0.21177028","-0.39043218","-0.20116656","-
0.3420598","-0.2818045","32"],
["33","6002091","Memory","2","9","Correct","-0.18970458","-]

0.3671769", "-0.32242432", "-0.31489053", "-0.28061774", "-0.26302513", "-
0.25742257", "-0.347411", "32"],
["34", "6003091", "Memory", "3", "9", "Correct", "-0.5318925", "-
0.082937084", "-0.51751447", "-0.18144473", "-0.58348364", "-
0.15934539", "-0.44372365", "-0.103356145", "32"],
["35", "6004090", "Memory", "4", "9", "Incorrect", "-0.30589694", "-
0.20537476", "-0.20996885", "-0.20663786", "-0.29256895", "-0.30830526", "-
0.22485064", "-0.21269903", "32"],
["36", "6005090", "Memory", "5", "9", "Incorrect", "0.12930058", "-
0.28820366", "-0.032250695", "-0.23420873", "0.016437883", "-
0.22717562", "-0.1290134", "-0.24308996", "32"],
["37", "6006090", "Memory", "6", "9", "Incorrect", "-0.26818398", "-
0.2829263", "-0.27913895", "-0.28561494", "-0.094576724", "-0.23920876", "-
0.3018723", "-0.25620952", "32"],
["38", "6007090", "Memory", "7", "9", "Incorrect", "0.084834255", "-
0.26567012", "-0.030220974", "-0.23396233", "-0.03297966", "-
0.24679531", "-0.050285477", "-0.22162485", "32"],
["39", "6008090", "Memory", "8", "9", "Incorrect", "-0.30791634", "-
0.20874251", "-0.22088756", "-0.18463473", "-0.2262393", "-0.14410287", "-
0.2781134", "-0.21224517", "32"],
["40", "6009090", "Memory", "9", "9", "Incorrect", "-0.4773843", "-
0.16819796", "-0.5214927", "-0.27013224", "-0.19666412", "-0.17574364", "-
0.41875088", "-0.19983512", "32"],
["41", "6001130", "Memory", "1", "13", "Incorrect", "-0.31084925", "-
0.31174663", "-0.48219556", "-0.2512052", "-0.5094922", "-0.23546682", "-
0.37655255", "-0.24399462", "32"],
["42", "6002130", "Memory", "2", "13", "Incorrect", "-0.2661904", "-
0.1507778", "-0.2572337", "-0.19883099", "-0.30389428", "-0.25442114", "-
0.2729734", "-0.1592433", "32"],
["43", "6003130", "Memory", "3", "13", "Incorrect", "-0.25882336", "-
0.21254426", "-0.21985504", "-0.24547085", "-0.09906214", "-0.18873982", "-
0.2514184", "-0.275629", "32"],
["44", "6004130", "Memory", "4", "13", "Incorrect", "-0.22194955", "-
0.19470334", "-0.20325668", "-0.19758986", "-0.31372896", "-0.17164063", "-
0.14343522", "-0.2219976", "32"],
["45", "6005130", "Memory", "5", "13", "Incorrect", "-0.12569916", "-
0.218465", "-0.195322", "-0.269256", "-0.058102686", "-0.28585035", "-
0.1949538", "-0.23295939", "32"],
["46", "6006130", "Memory", "6", "13", "Incorrect", "-0.32170615", "-
0.21977203", "-0.35010326", "-0.21004836", "-0.34078032", "-0.1637219", "-
0.31179363", "-0.22396618", "32"],
["47", "6007130", "Memory", "7", "13", "Incorrect", "-0.24178806", "-
0.23869379", "-0.22467299", "-0.22116794", "-0.29009837", "-0.22516188", "-
0.11503375", "-0.22749622", "32"],
["48", "6008130", "Memory", "8", "13", "Incorrect", "-0.03869223", "-
0.30358636", "-0.1122055", "-0.22189201", "-0.13732743", "-0.2437571", "-
0.09427635", "-0.25878254", "32"],
["49", "6009130", "Memory", "9", "13", "Incorrect", "-0.2647155", "-


```
0.121729136", "-0.22190028", "-0.19463888", "-0.19198294", "-0.1838097", "-  
0.21076103", "-0.14502469", "32"]], "shape": {"columns": 14, "rows": 29187}}
```

```
X = df_features_2[["frontal_alpha", "frontal_gamma",  
    "parietal_alpha", "parietal_gamma",  
    "temporal_alpha", "temporal_gamma",  
    "midline_alpha", "midline_gamma"]  
]]
```

```
Y = df_features_2["condition"]  
group = df_features_2["participant"]
```

```
display(X)  
display(Y)  
display(group)
```

```
{"columns": [{"name": "index", "rawType": "int64", "type": "integer"},  
{"name": "frontal_alpha", "rawType": "float64", "type": "float"},  
{"name": "frontal_gamma", "rawType": "float64", "type": "float"},  
{"name": "parietal_alpha", "rawType": "float64", "type": "float"},  
{"name": "parietal_gamma", "rawType": "float64", "type": "float"},  
{"name": "temporal_alpha", "rawType": "float64", "type": "float"},  
{"name": "temporal_gamma", "rawType": "float64", "type": "float"},  
{"name": "midline_alpha", "rawType": "float64", "type": "float"},  
{"name": "midline_gamma", "rawType": "float64", "type": "float"}], "ref": "fe  
b9360a-e522-457e-9e8e-840958823016", "rows": [{"0", "-0.025957057", "-  
0.18326485", "-0.009031365", "-0.20584455", "-0.15735893", "-  
0.19449925", "-0.09644044", "-0.16075216"}, {"1", "-0.4295599", "-  
0.1586829", "-0.39929757", "-0.2581144", "-0.36366075", "-0.1721333", "-  
0.40252006", "-0.16622323"}, {"2", "-0.42290002", "-0.24490291", "-  
0.4255417", "-0.17459345", "-0.30750823", "-0.2257737", "-0.41035983", "-  
0.22974303"}, {"3", "-0.06936548", "-0.21740115", "-0.060178876", "-  
0.20385209", "0.06379208", "-0.21597047", "-0.11895034", "-0.21043433"},  
{"4", "-0.30864483", "-0.2719169", "-0.2301973", "-0.2210616", "-  
0.36474884", "-0.2684594", "-0.2772169", "-0.25362664"}, {"5", "-  
0.28857574", "-0.19293183", "-0.16426952", "-0.21656498", "-0.2809203", "-  
0.16254508", "-0.21992949", "-0.18849497"}, {"6", "-0.20350565", "-  
0.21727931", "-0.32124722", "-0.24504736", "-0.1468042", "-0.22221668", "-  
0.30233803", "-0.27994496"}, {"7", "-0.34081808", "-0.22551022", "-  
0.45709014", "-0.22024778", "-0.41842172", "-0.23807715", "-0.3539373", "-  
0.20218052"}, {"8", "-0.025039962", "-0.18180633", "-0.082695514", "-  
0.22573274", "-0.06977223", "-0.27738345", "-0.080132455", "-0.09608986"},  
{"9", "-0.22259584", "-0.32267013", "-0.2140352", "-0.37525222", "-  
0.22824852", "-0.33167717", "-0.1656693", "-0.29273036"}, {"10", "-  
0.09276553", "-0.2743676", "-0.049321573", "-0.20778574", "-  
0.089088164", "-0.2447739", "-0.10138233", "-0.32953507"}, {"11", "-  
0.9575961", "-0.3040309", "-0.691751", "-0.22905163", "-0.6792006", "-  
0.2936776", "-0.7334833", "-0.26064476"}, {"12", "-0.42778593", "-  
0.21990559", "-0.2722173", "-0.2260552", "-0.48790473", "-0.2311506", "-  
0.32097512", "-0.17965907"}, {"13", "-0.27830294", "-0.6870689", "-
```

0.23974974", "-0.42242566", "-0.44163892", "-0.7199085", "-0.39450857", "-
0.37641114"], ["14", "-0.34897283", "-0.31278422", "-0.3084698", "-
0.29483205", "-0.41784438", "-0.26977026", "-0.25131318", "-0.30931133"],
["15", "-0.2729121", "-0.06319353", "-0.0009827339", "-
0.063633464", "0.05674658", "-0.1197799", "-0.07165969", "-0.14079067"],
["16", "-0.063734435", "-0.27875915", "-0.33953163", "-0.34841815", "-
0.33590022", "-0.28846663", "-0.19536875", "-0.29774624"],
["17", "0.100504465", "-0.29090923", "-0.092323095", "-
0.2601843", "0.08237771", "-0.29721344", "-0.014801744", "-0.145563"],
["18", "-0.11886485", "-0.15496579", "-0.14856766", "-0.23000282", "-
0.21635072", "-0.23587619", "-0.1684342", "-0.11889206"], ["19", "-
0.33948734", "-0.2840895", "-0.35501134", "-0.25531724", "-0.51224536", "-
0.2544201", "-0.27235836", "-0.27527538"], ["20", "-0.012826612", "-
0.30882943", "0.022352863", "-0.25369757", "0.034773197", "-0.27630496", "-
0.13377067", "-0.26401532"], ["21", "-0.05910009", "-0.17341319", "-
0.094559416", "-0.14685845", "-0.12642705", "-0.20388396", "-
0.12183217", "-0.19637032"], ["22", "-0.5474208", "-0.13229024", "-
0.4890003", "-0.09876043", "-0.6448048", "-0.1219852", "-0.6356747", "-
0.14665087"], ["23", "-0.2589021", "-0.21471645", "-0.16241616", "-
0.23287363", "-0.0867925", "-0.2541058", "-0.2463156", "-0.17246994"],
["24", "-0.1014311", "-0.22763164", "-0.112267636", "-0.23234826", "-
0.18075179", "-0.24761663", "-0.12586412", "-0.26313964"], ["25", "-
0.03089234", "-0.2904603", "-0.3292658", "-0.27815068", "-0.1869417", "-
0.13189666", "-0.09611091", "-0.26322964"], ["26", "-0.055112306", "-
0.1931216", "0.015092393", "-0.2610998", "0.120639585", "-0.15700145", "-
0.03282554", "-0.20202817"], ["27", "0.19306137", "-
0.081946254", "0.015768766", "-0.157055", "0.07461289", "-
0.20597205", "0.05658468", "-0.12851547"], ["28", "0.03840608", "-
0.3173205", "0.009145574", "-0.19637242", "-0.20130816", "-0.3014595", "-
0.20558067", "-0.19098456"], ["29", "-0.29856443", "-0.28146115", "-
0.29204983", "-0.23172358", "-0.31965402", "-0.25834638", "-0.30473813", "-
0.33122146"], ["30", "-0.2702053", "-0.24439706", "-0.22737053", "-
0.27406314", "-0.29719052", "-0.22277696", "-0.21135351", "-0.25489524"],
["31", "-0.47628137", "-0.21425423", "-0.4897416", "-0.2394687", "-
0.3333102", "-0.19943354", "-0.42209068", "-0.18647537"], ["32", "-
0.35371333", "-0.21229637", "-0.40571758", "-0.21177028", "-0.39043218", "-
0.20116656", "-0.3420598", "-0.2818045"], ["33", "-0.18970458", "-
0.3671769", "-0.32242432", "-0.31489053", "-0.28061774", "-0.26302513", "-
0.25742257", "-0.347411"], ["34", "-0.5318925", "-0.082937084", "-
0.51751447", "-0.18144473", "-0.58348364", "-0.15934539", "-0.44372365", "-
0.103356145"], ["35", "-0.30589694", "-0.20537476", "-0.20996885", "-
0.20663786", "-0.29256895", "-0.30830526", "-0.22485064", "-0.21269903"],
["36", "0.12930058", "-0.28820366", "-0.032250695", "-
0.23420873", "0.016437883", "-0.22717562", "-0.1290134", "-0.24308996"],
["37", "-0.26818398", "-0.2829263", "-0.27913895", "-0.28561494", "-
0.094576724", "-0.23920876", "-0.3018723", "-0.25620952"],
["38", "0.084834255", "-0.26567012", "-0.030220974", "-0.23396233", "-
0.03297966", "-0.24679531", "-0.050285477", "-0.22162485"], ["39", "-
0.30791634", "-0.20874251", "-0.22088756", "-0.18463473", "-0.2262393", "-

```

0.14410287", "-0.2781134", "-0.21224517"], ["40", "-0.4773843", "-
0.16819796", "-0.5214927", "-0.27013224", "-0.19666412", "-0.17574364", "-
0.41875088", "-0.19983512"], ["41", "-0.31084925", "-0.31174663", "-
0.48219556", "-0.2512052", "-0.5094922", "-0.23546682", "-0.37655255", "-
0.24399462"], ["42", "-0.2661904", "-0.1507778", "-0.2572337", "-
0.19883099", "-0.30389428", "-0.25442114", "-0.2729734", "-0.1592433"],
["43", "-0.25882336", "-0.21254426", "-0.21985504", "-0.24547085", "-
0.09906214", "-0.18873982", "-0.2514184", "-0.275629"], ["44", "-
0.22194955", "-0.19470334", "-0.20325668", "-0.19758986", "-0.31372896", "-
0.17164063", "-0.14343522", "-0.2219976"], ["45", "-0.12569916", "-
0.218465", "-0.195322", "-0.269256", "-0.058102686", "-0.28585035", "-
0.1949538", "-0.23295939"], ["46", "-0.32170615", "-0.21977203", "-
0.35010326", "-0.21004836", "-0.34078032", "-0.1637219", "-0.31179363", "-
0.22396618"], ["47", "-0.24178806", "-0.23869379", "-0.22467299", "-
0.22116794", "-0.29009837", "-0.22516188", "-0.11503375", "-0.22749622"],
["48", "-0.03869223", "-0.30358636", "-0.1122055", "-0.22189201", "-
0.13732743", "-0.2437571", "-0.09427635", "-0.25878254"], ["49", "-
0.2647155", "-0.121729136", "-0.22190028", "-0.19463888", "-0.19198294", "-
0.1838097", "-0.21076103", "-0.14502469"]], "shape":
{"columns":8,"rows":29187}}

```

```

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"condition","rawType":"str","type":"string"}],"ref":"d6fdb108-
349f-4bcf-a041-73ff1690a0b4","rows":[["0","Listen"],["1","Listen"],
["2","Listen"],["3","Listen"],["4","Listen"],["5","Listen"],
["6","Listen"],["7","Listen"],["8","Listen"],["9","Listen"],
["10","Listen"],["11","Listen"],["12","Listen"],["13","Listen"],
["14","Listen"],["15","Listen"],["16","Listen"],["17","Listen"],
["18","Listen"],["19","Listen"],["20","Listen"],["21","Listen"],
["22","Listen"],["23","Listen"],["24","Listen"],["25","Listen"],
["26","Listen"],["27","Memory"],["28","Memory"],["29","Memory"],
["30","Memory"],["31","Memory"],["32","Memory"],["33","Memory"],
["34","Memory"],["35","Memory"],["36","Memory"],["37","Memory"],
["38","Memory"],["39","Memory"],["40","Memory"],["41","Memory"],
["42","Memory"],["43","Memory"],["44","Memory"],["45","Memory"],
["46","Memory"],["47","Memory"],["48","Memory"],
["49","Memory"]],"shape":{"columns":1,"rows":29187}}

```

```

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"participant","rawType":"int64","type":"integer"}],"ref":"b3ac
c13a-aa25-473d-a96f-5dd14875e908","rows":[["0","32"],["1","32"],
["2","32"],["3","32"],["4","32"],["5","32"],["6","32"],["7","32"],
["8","32"],["9","32"],["10","32"],["11","32"],["12","32"],["13","32"],
["14","32"],["15","32"],["16","32"],["17","32"],["18","32"],
["19","32"],["20","32"],["21","32"],["22","32"],["23","32"],
["24","32"],["25","32"],["26","32"],["27","32"],["28","32"],
["29","32"],["30","32"],["31","32"],["32","32"],["33","32"],
["34","32"],["35","32"],["36","32"],["37","32"],["38","32"],
["39","32"],["40","32"],["41","32"],["42","32"],["43","32"],

```

```
[ "44", "32"], ["45", "32"], ["46", "32"], ["47", "32"], ["48", "32"],  
["49", "32"]], "shape": {"columns": 1, "rows": 29187}}
```

Hyperparameter tuning

```
logo = LeaveOneGroupOut()  
  
param_grid = {  
    "n_estimators" : [50, 100, 200, 300, 400],  
    "max_depth" : [10, 15, 20, 25, 30],  
    "min_samples_split" : [2, 5, 10, 15, 20, 25]  
}  
  
random_search = RandomizedSearchCV(  
    RandomForestClassifier(random_state=42, class_weight="balanced"),  
    param_distributions = param_grid,  
    n_iter = 25,  
    cv = list(logo.split(X, Y, groups=group)),  
    scoring = "f1_macro",  
    random_state = 42,  
    n_jobs = -1,  
)  
  
random_search.fit(X, Y)  
best_params = random_search.best_params_  
print(f"The best params are: {best_params}")  
print(f"The best score is: {random_search.best_score}")  
  
The best params are: {'n_estimators': 400, 'min_samples_split': 15,  
'max_depth': 10}  
The best score is: 0.5152142453829789
```

Hyperparameter tuning conclusion

With n=25 run, I got:

- The best params are: {'n_estimators': 400, 'min_samples_split': 15, 'max_depth': 10}
- The best score is: 0.5152142453829789

I will use this `best_params` in running my random forest model.

Running my random forest

```
best_params = random_search.best_params_  
accuracies, true, predicted = [], [], []  
accuracy  
for train, test in logo.split(X, Y, groups=group):  
    classifier = RandomForestClassifier(**best_params,  
    random_state=42, class_weight="balanced")  
    classifier.fit(X.iloc[train], Y.iloc[train])  
    prediction = classifier.predict(X.iloc[test])
```

```

accuracy = accuracy_score(Y.iloc[test], prediction)
accuracies.append(accuracy)
true.extend(Y.iloc[test])
predicted.extend(prediction)

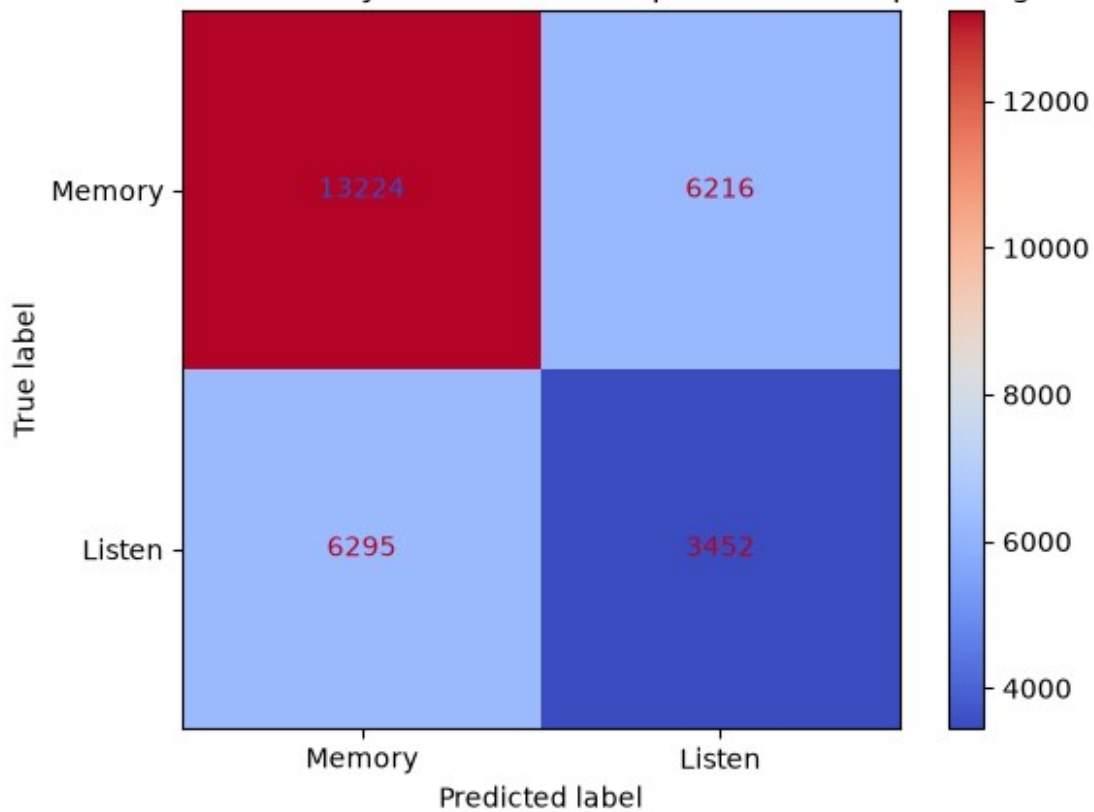
cm = confusion_matrix(true, predicted, labels=["Memory", "Listen"])
ConfusionMatrixDisplay(cm, display_labels=["Memory",
"Listen"]).plot(cmap="coolwarm")
plt.title("Confusion Matrix: Memory vs Listen - ROI specific brain
alpha vs gamma")
plt.show()

report = classification_report(true, predicted, output_dict = True)
report_df = pd.DataFrame(report).transpose().round(3)
display(report_df)

scores_df.loc[len(scores_df)] = ["our-model-2", "accuracy", accuracy]
scores_df.loc[len(scores_df)] = ["our-model-2", "f1-macro",
report_df.loc["macro avg", "f1-score"]]
scores_df.loc[len(scores_df)] = ["our-model-2", "f1-avg",
report_df.loc["weighted avg", "f1-score"]]
display(scores_df)

```

Confusion Matrix: Memory vs Listen - ROI specific brain alpha vs gamma



```
{
  "columns": [
    {"name": "index", "rawType": "str", "type": "string"},
    {"name": "precision", "rawType": "float64", "type": "float"},
    {"name": "recall", "rawType": "float64", "type": "float"},
    {"name": "f1-score", "rawType": "float64", "type": "float"},
    {"name": "support", "rawType": "float64", "type": "float"}
  ],
  "ref": "26741e4b-2fdd-4f9e-a098-98cb6086aa3c",
  "rows": [
    ["Listen", "0.357", "0.354", "0.356", "9747.0"],
    ["Memory", "0.677", "0.68", "0.679", "19440.0"],
    ["accuracy", "0.571", "0.571", "0.571", "0.571"],
    ["macro avg", "0.517", "0.517", "0.517", "29187.0"],
    ["weighted avg", "0.57", "0.571", "0.571", "29187.0"]
  ],
  "shape": {
    "columns": 4,
    "rows": 5
  }
}

{
  "columns": [
    {"name": "index", "rawType": "int64", "type": "integer"},
    {"name": "model", "rawType": "str", "type": "string"},
    {"name": "metric", "rawType": "str", "type": "string"},
    {"name": "score", "rawType": "float64", "type": "float"}
  ],
  "ref": "adc184fe-ba78-40c5-abce-94236d84697f",
  "rows": [
    ["0", "our-model", "accuracy", "0.5727023319615913"],
    ["1", "our-model", "f1-macro", "0.513"],
    ["2", "our-model", "f1-avg", "0.561"],
    ["3", "dummy-most-frequent", "accuracy", "0.6666666666666666"],
    ["4", "dummy-most-frequent", "f1-macro", "0.4"],
    ["5", "dummy-most-frequent", "f1-avg", "0.533"],
    ["6", "dummy-stratified", "accuracy", "0.6666666666666666"],
    ["7", "dummy-stratified", "f1-macro", "0.501"],
    ["8", "dummy-stratified", "f1-avg", "0.556"],
    ["9", "our-model-2", "accuracy", "0.578875171467764"],
    ["10", "our-model-2", "f1-macro", "0.517"],
    ["11", "our-model-2", "f1-avg", "0.571"]
  ],
  "shape": {
    "columns": 3,
    "rows": 12
  }
}
```

Comparing with whole brain alpha-gamma

```
comparison_df = scores_df[scores_df["model"].isin(["our-model", "our-model-2"])]

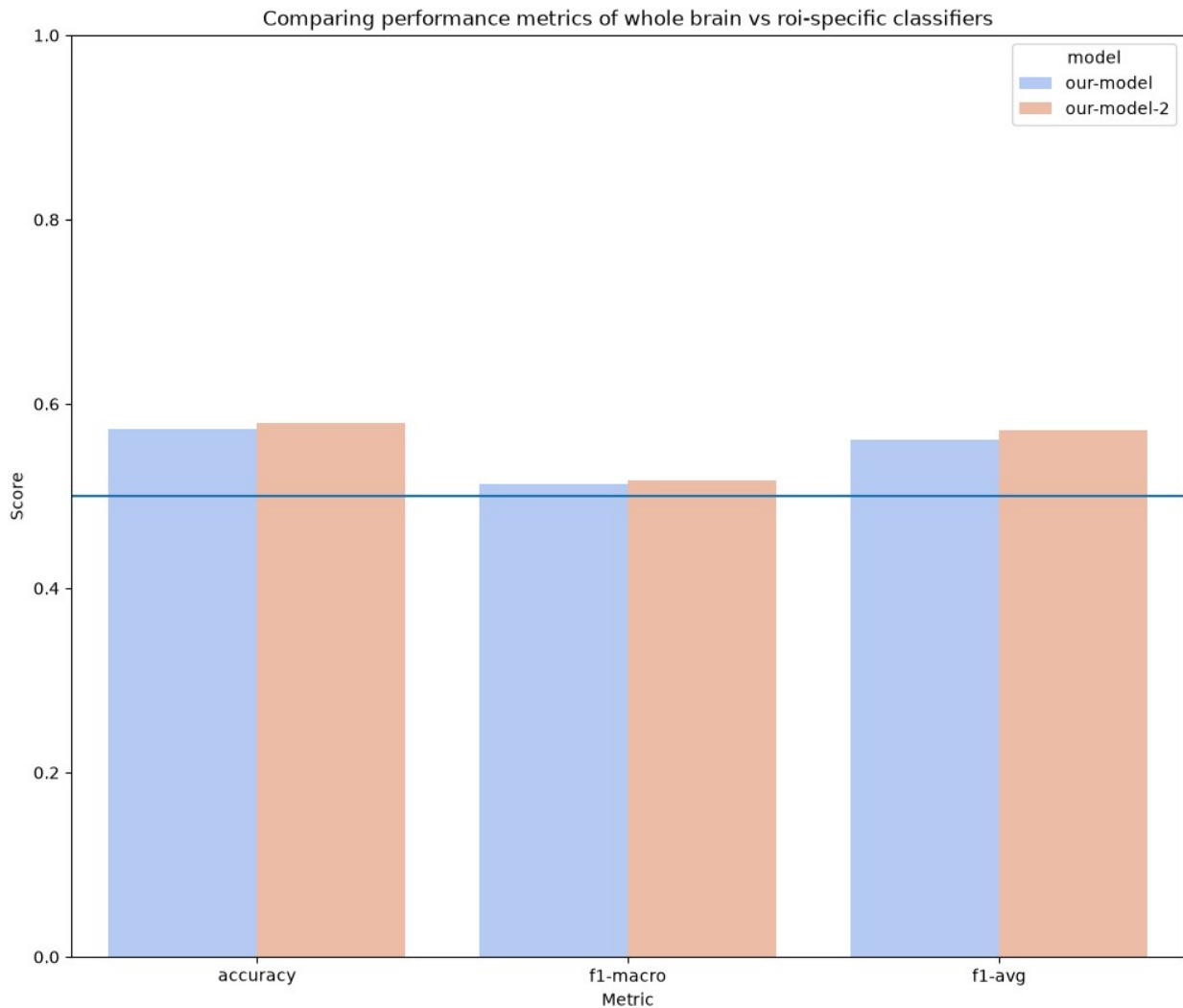
display(comparison_df)

plt.figure(figsize=(12, 10))
sns.barplot(x="metric", y="score", hue="model", data=comparison_df, palette="coolwarm")
plt.title("Comparing performance metrics of whole brain vs roi-specific classifiers")
plt.xlabel("Metric")
plt.ylabel("Score")
plt.axhline(0.5)
plt.ylim(0, 1)

{
  "columns": [
    {"name": "index", "rawType": "int64", "type": "integer"},
    {"name": "model", "rawType": "str", "type": "string"},
    {"name": "metric", "rawType": "str", "type": "string"},
    {"name": "score", "rawType": "float64", "type": "float"}
  ],
  "ref": "a075ce4d-
```

```
f048-46d2-8354-cee9de00dc4c", "rows": [{"0", "our-model", "accuracy", "0.5727023319615913"}, {"1", "our-model", "f1-macro", "0.513"}, {"2", "our-model", "f1-avg", "0.561"}, {"9", "our-model-2", "accuracy", "0.578875171467764"}, {"10", "our-model-2", "f1-macro", "0.517"}, {"11", "our-model-2", "f1-avg", "0.571"}], "shape": {"columns": 3, "rows": 6}}
```

(0.0, 1.0)

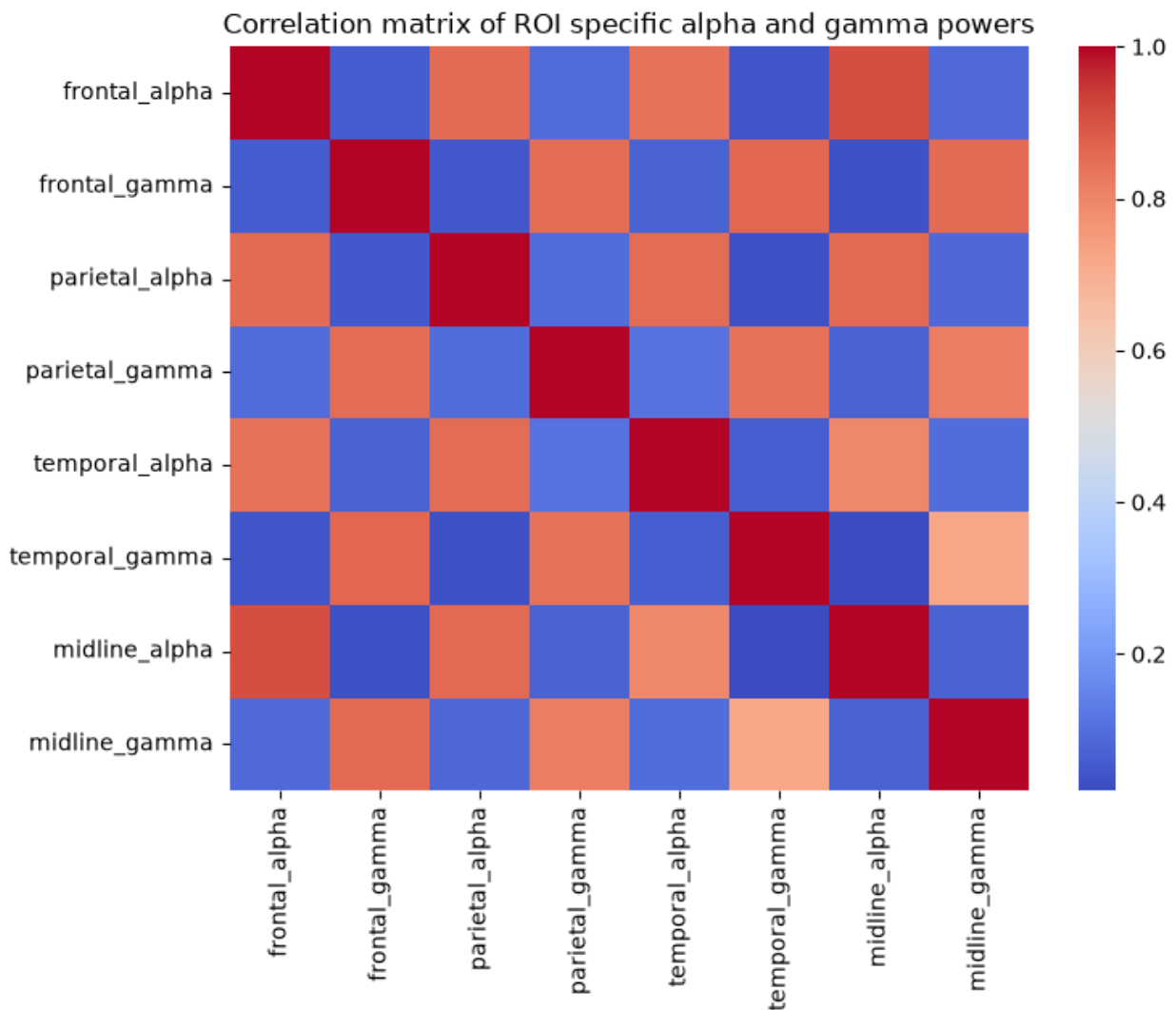


Markdown conclusion

As we see above, our ROI specific model performs slightly better than a whole-brain model likely because each brain region performs differently in different tasks. However, our correlation matrix below shows strong correlation between each region, thus, we still cannot reliably determine which condition the participant is in from ROI specific wave power alone. The boxplot also shows that there is not enough difference between ROI-specific wave powers across conditions to reliably create a model to determine condition from EEG.

Diagnosing model performance

```
corr_matrix = df_features_2[["frontal_alpha", "frontal_gamma",  
    "parietal_alpha", "parietal_gamma",  
    "temporal_alpha", "temporal_gamma",  
    "midline_alpha", "midline_gamma"]].corr()  
  
plt.figure(figsize=(8,6))  
sns.heatmap(corr_matrix, cmap="coolwarm", fmt=".2f")  
plt.title("Correlation matrix of ROI specific alpha and gamma powers")  
plt.show()
```



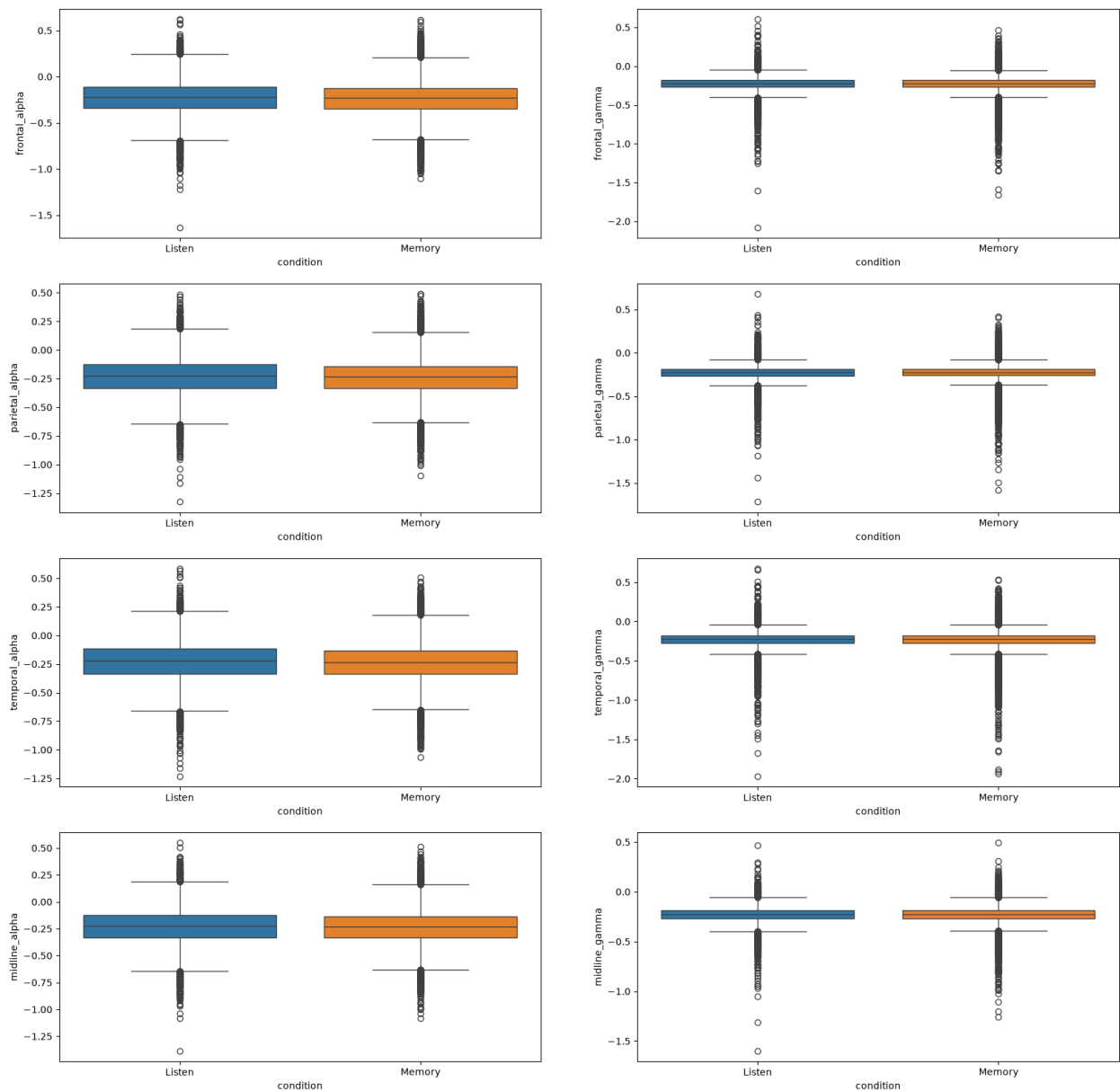
```
waves = ["alpha", "gamma"]  
regions = ["frontal", "parietal", "temporal", "midline"]  
fig, ax = plt.subplots(4, 2, figsize=(20,20))
```

```

for row, region in enumerate(regions):
    for column, wave in enumerate(waves):
        column_name = f"{region}_{wave}"
        sns.boxplot(x="condition", y=column_name, ax=ax[row, column],
data=df_features_2, hue="condition")
plt.suptitle("Differences in ROI specific alpha and gamma power by
condition", fontsize=20)
plt.show()

```

Differences in ROI specific alpha and gamma power by condition



[NOT USEFUL]: Classify: Determine if-correct from ROI specific alpha/gamma and load - almost completely dominated by load

Load data

```
memory_only = df_features_2[df_features_2["condition"] ==
"Memory"].copy()

X = memory_only[["frontal_alpha", "frontal_gamma",
    "parietal_alpha", "parietal_gamma",
    "temporal_alpha", "temporal_gamma",
    "midline_alpha", "midline_gamma"
] + ["load"]]

Y = memory_only["correct"]
group = memory_only["participant"]

display(X)
display(Y)
display(group)

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"frontal_alpha","rawType":"float64","type":"float"},
{"name":"frontal_gamma","rawType":"float64","type":"float"},
{"name":"parietal_alpha","rawType":"float64","type":"float"},
{"name":"parietal_gamma","rawType":"float64","type":"float"},
{"name":"temporal_alpha","rawType":"float64","type":"float"},
{"name":"temporal_gamma","rawType":"float64","type":"float"},
{"name":"midline_alpha","rawType":"float64","type":"float"},
{"name":"midline_gamma","rawType":"float64","type":"float"},
{"name":"load","rawType":"int64","type":"integer"}],"ref":"a455a7c5-2d53-44be-8bef-63225e007f7b","rows":[["27","0.19306137","-0.081946254","0.015768766","-0.157055","0.07461289","-0.20597205","0.05658468","-0.12851547","5"],["28","0.03840608","-0.3173205","0.009145574","-0.19637242","-0.20130816","-0.3014595","-0.20558067","-0.19098456","5"],["29","-0.29856443","-0.28146115","-0.29204983","-0.23172358","-0.31965402","-0.25834638","-0.30473813","-0.33122146","5"],["30","-0.2702053","-0.24439706","-0.22737053","-0.27406314","-0.29719052","-0.22277696","-0.21135351","-0.25489524","5"],["31","-0.47628137","-0.21425423","-0.4897416","-0.2394687","-0.3333102","-0.19943354","-0.42209068","-0.18647537","5"],["32","-0.35371333","-0.21229637","-0.40571758","-0.21177028","-0.39043218","-0.20116656","-0.3420598","-0.2818045","9"],["33","-0.18970458","-0.3671769","-0.32242432","-0.31489053","-0.28061774","-0.26302513","-0.25742257","-0.347411","9"],["34","-0.5318925","-0.082937084","-0.51751447","-0.18144473","-0.58348364","-0.15934539","-0.44372365","-0.103356145","9"],["35","-0.30589694","-0.20537476","-0.20996885","-
```

0.20663786", "-0.29256895", "-0.30830526", "-0.22485064", "-
0.21269903", "9"], ["36", "0.12930058", "-0.28820366", "-0.032250695", "-
0.23420873", "0.016437883", "-0.22717562", "-0.1290134", "-
0.24308996", "9"], ["37", "-0.26818398", "-0.2829263", "-0.27913895", "-
0.28561494", "-0.094576724", "-0.23920876", "-0.3018723", "-
0.25620952", "9"], ["38", "0.084834255", "-0.26567012", "-0.030220974", "-
0.23396233", "-0.03297966", "-0.24679531", "-0.050285477", "-
0.22162485", "9"], ["39", "-0.30791634", "-0.20874251", "-0.22088756", "-
0.18463473", "-0.2262393", "-0.14410287", "-0.2781134", "-
0.21224517", "9"], ["40", "-0.4773843", "-0.16819796", "-0.5214927", "-
0.27013224", "-0.19666412", "-0.17574364", "-0.41875088", "-
0.19983512", "9"], ["41", "-0.31084925", "-0.31174663", "-0.48219556", "-
0.2512052", "-0.5094922", "-0.23546682", "-0.37655255", "-
0.24399462", "13"], ["42", "-0.2661904", "-0.1507778", "-0.2572337", "-
0.19883099", "-0.30389428", "-0.25442114", "-0.2729734", "-
0.1592433", "13"], ["43", "-0.25882336", "-0.21254426", "-0.21985504", "-
0.24547085", "-0.09906214", "-0.18873982", "-0.2514184", "-
0.275629", "13"], ["44", "-0.22194955", "-0.19470334", "-0.20325668", "-
0.19758986", "-0.31372896", "-0.17164063", "-0.14343522", "-
0.2219976", "13"], ["45", "-0.12569916", "-0.218465", "-0.195322", "-
0.269256", "-0.058102686", "-0.28585035", "-0.1949538", "-
0.23295939", "13"], ["46", "-0.32170615", "-0.21977203", "-0.35010326", "-
0.21004836", "-0.34078032", "-0.1637219", "-0.31179363", "-
0.22396618", "13"], ["47", "-0.24178806", "-0.23869379", "-0.22467299", "-
0.22116794", "-0.29009837", "-0.22516188", "-0.11503375", "-
0.22749622", "13"], ["48", "-0.03869223", "-0.30358636", "-0.1122055", "-
0.22189201", "-0.13732743", "-0.2437571", "-0.09427635", "-
0.25878254", "13"], ["49", "-0.2647155", "-0.121729136", "-0.22190028", "-
0.19463888", "-0.19198294", "-0.1838097", "-0.21076103", "-
0.14502469", "13"], ["50", "-0.15205804", "-0.20811442", "-0.21949725", "-
0.26230317", "-0.10198443", "-0.26996976", "-0.28665683", "-
0.17182894", "13"], ["51", "-0.33034128", "-0.2753052", "-0.43461314", "-
0.18113676", "-0.41043055", "-0.19181193", "-0.32377246", "-
0.29798153", "13"], ["52", "-0.47582638", "-0.26470008", "-0.33637944", "-
0.19597386", "-0.40165198", "-0.19760264", "-0.40935275", "-
0.24135403", "13"], ["53", "-0.27548108", "-0.20644328", "-0.2105312", "-
0.14705351", "-0.17359707", "-0.20049216", "-0.32881108", "-
0.109784424", "13"], ["54", "-0.2959641", "-0.28914094", "-0.38326168", "-
0.26323712", "-0.3825686", "-0.2799678", "-0.38195205", "-0.2514945", "5"],
["55", "-0.14230321", "-0.18829356", "-0.0063392473", "-
0.2578714", "0.005966615", "-0.25752795", "-0.11230832", "-
0.20093521", "5"], ["56", "-0.27317438", "-0.28515536", "-0.107199036", "-
0.1951966", "-0.13444282", "-0.2771769", "-0.23082113", "-
0.27872193", "5"], ["57", "-0.39714694", "-0.19032684", "-0.4302017", "-
0.23461118", "-0.41604266", "-0.12822938", "-0.38801602", "-
0.17935817", "5"], ["58", "-0.13435309", "-0.24434043", "-0.2347351", "-
0.21788888", "-0.2135736", "-0.26003468", "-0.19979118", "-
0.22541891", "5"], ["59", "0.11957043", "-0.2451952", "0.18963426", "-
0.22826454", "0.18634252", "-0.20386246", "0.15909839", "-0.2935357", "9"],

```
[ "60", "-0.2621587", "-0.3024373", "-0.09144223", "-0.24720298", "-0.12650849", "-0.23790152", "-0.12833303", "-0.2456428", "9"], [ "61", "-0.38387498", "-0.24394555", "-0.47699305", "-0.24638528", "-0.4480101", "-0.2491558", "-0.37287688", "-0.18384415", "9"], [ "62", "-0.6077923", "-0.21970813", "-0.62148505", "-0.2934628", "-0.590291", "-0.30022398", "-0.6305598", "-0.31197333", "9"], [ "63", "-0.09525991", "-0.2551221", "-0.22097033", "-0.21886784", "-0.09266916", "-0.2354942", "-0.20380133", "-0.22873971", "9"], [ "64", "-0.06488484", "-0.22547358", "-0.21952966", "-0.22767001", "-0.17938364", "-0.29820424", "-0.05166395", "-0.19273831", "9"], [ "65", "0.08251101", "-0.25464278", "0.04425769", "-0.19865315", "0.06301021", "-0.22130355", "-0.012882215", "-0.25250086", "9"], [ "66", "-0.17742155", "-0.2238608", "-0.098720625", "-0.19353019", "-0.23671703", "-0.18961012", "-0.22685865", "-0.22684245", "9"], [ "67", "-0.16681392", "-0.18191507", "-0.2557338", "-0.24696209", "-0.2058761", "-0.27194238", "-0.19087201", "-0.1934232", "9"], [ "68", "-0.25772625", "-0.21678737", "-0.15528092", "-0.23221071", "-0.30774167", "-0.16282015", "-0.16455191", "-0.18829645", "9"], [ "69", "0.08875839", "-0.20816782", "0.123848476", "-0.23878846", "-0.028185703", "-0.18096718", "0.032135665", "-0.21407776", "9"], [ "70", "-0.6446771", "-0.29819992", "-0.5034082", "-0.3196487", "-0.54998577", "-0.28294694", "-0.5944375", "-0.2938728", "9"], [ "71", "-0.23400283", "-0.2359358", "-0.32505903", "-0.22115697", "-0.29564962", "-0.2634722", "-0.25406006", "-0.1868799", "9"], [ "72", "-0.03758158", "-0.18486021", "-0.18876784", "-0.16475913", "-0.04619868", "-0.23223592", "-0.13120794", "-0.12697998", "9"], [ "73", "-0.30098143", "-0.21866086", "-0.35391146", "-0.18464096", "-0.26279584", "-0.24733487", "-0.26273474", "-0.18505323", "9"], [ "74", "-0.18243586", "-0.16215162", "-0.16650221", "-0.23670718", "-0.18130597", "-0.1654397", "-0.2132104", "-0.21317972", "9"], [ "75", "-0.36494407", "-0.21607347", "-0.35675588", "-0.23415521", "-0.454927", "-0.28045017", "-0.26812762", "-0.19431876", "9"], [ "76", "-0.223572", "-0.19547921", "-0.32643598", "-0.20583162", "-0.19378714", "-0.18368755", "-0.3298045", "-0.21630292", "9"]], "shape": {"columns": 9, "rows": 19440}}
```

```
{"columns": [{"name": "index", "rawType": "int64", "type": "integer"}, {"name": "correct", "rawType": "str", "type": "string"}], "ref": "3e27de60-d774-4fb5-a2d4-c5cd6864293c", "rows": [{"27", "Correct"}, {"28", "Correct"}, {"29", "Correct"}, {"30", "Correct"}, {"31", "Correct"}, {"32", "Correct"}, {"33", "Correct"}, {"34", "Correct"}, {"35", "Incorrect"}, {"36", "Incorrect"}, {"37", "Incorrect"}, {"38", "Incorrect"}, {"39", "Incorrect"}, {"40", "Incorrect"}, {"41", "Incorrect"}, {"42", "Incorrect"}, {"43", "Incorrect"}, {"44", "Incorrect"}, {"45", "Incorrect"}, {"46", "Incorrect"}, {"47", "Incorrect"}, {"48", "Incorrect"}, {"49", "Incorrect"}, {"50", "Incorrect"}, {"51", "Incorrect"}, {"52", "Incorrect"}, {"53", "Incorrect"}, {"54", "Correct"}, {"55", "Correct"}, {"56", "Correct"}, {"57", "Correct"}, {"58", "Correct"}, {"59", "Correct"}, {"60", "Correct"}, {"61", "Correct"}, {"62", "Correct"}, {"63", "Correct"}, {"64", "Correct"}, {"65", "Incorrect"}, {"66", "Incorrect"}, {"67", "Incorrect"}, {"68", "Incorrect"}, {"69", "Correct"}, {"70", "Correct"}, {"71", "Correct"}, {"72", "Incorrect"}],
```

```

["73","Incorrect"],["74","Incorrect"],["75","Incorrect"],
["76","Incorrect"]], "shape":{"columns":1,"rows":19440}}

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"participant","rawType":"int64","type":"integer"}],"ref":"266a
bb2e-c30c-4d24-a143-7bade4a2aba6","rows":[["27","32"],["28","32"],
["29","32"],["30","32"],["31","32"],["32","32"],["33","32"],
["34","32"],["35","32"],["36","32"],["37","32"],["38","32"],
["39","32"],["40","32"],["41","32"],["42","32"],["43","32"],
["44","32"],["45","32"],["46","32"],["47","32"],["48","32"],
["49","32"],["50","32"],["51","32"],["52","32"],["53","32"],
["54","32"],["55","32"],["56","32"],["57","32"],["58","32"],
["59","32"],["60","32"],["61","32"],["62","32"],["63","32"],
["64","32"],["65","32"],["66","32"],["67","32"],["68","32"],
["69","32"],["70","32"],["71","32"],["72","32"],["73","32"],
["74","32"],["75","32"],["76","32"]],"shape":
{"columns":1,"rows":19440}}

```

Hyperparameter tuning

```

logo = LeaveOneGroupOut()

param_grid = {
    "n_estimators" : [50, 100, 200, 300, 400],
    "max_depth" : [10, 15, 20, 25, 30],
    "min_samples_split" : [2, 5, 10, 15, 20, 25]
}

random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42, class_weight="balanced"),
    param_distributions = param_grid,
    n_iter = 25,
    cv = list(logo.split(X, Y, groups=group)),
    scoring = "f1_macro",
    random_state = 42,
    n_jobs = -1,
)

random_search.fit(X, Y)
best_params = random_search.best_params_
print(f"The best params are: {best_params}")
print(f"The best score is: {random_search.best_score_}")

The best params are: {'n_estimators': 100, 'min_samples_split': 5,
'max_depth': 15}
The best score is: 0.6699103762855374

```

Running random forest

```
best_params = random_search.best_params_  
accuracies, true, predicted = [], [], []  
feature_importances = []  
  
for train, test in logo.split(X, Y, groups=group):  
    classifier = RandomForestClassifier(**best_params,  
random_state=42, class_weight="balanced")  
    classifier.fit(X.iloc[train], Y.iloc[train])  
    prediction = classifier.predict(X.iloc[test])  
    accuracy = accuracy_score(Y.iloc[test], prediction)  
    accuracies.append(accuracy)  
    true.extend(Y.iloc[test])  
    predicted.extend(prediction)  
    feature_importances.append(classifier.feature_importances_)  
  
mean = np.mean(feature_importances, axis=0)  
stddev = np.mean(feature_importances, axis=0)  
  
print(f"Mean accuracy: {np.mean(accuracies)}")  
print(f"Classification report: {classification_report(true,  
predicted)}")  
feature_importances_df = pd.DataFrame({  
    "feature" : X.columns,  
    "mean" : mean,  
    "std. dev" : stddev  
})  
  
display(feature_importances_df)  
  
Mean accuracy: 0.6885802469135802  
Classification report:          precision    recall  f1-score  
support  
  
      Correct      0.75      0.50      0.60      8999  
    Incorrect      0.66      0.85      0.75     10441  
  
      accuracy          0.69     19440  
    macro avg      0.70      0.68      0.67     19440  
weighted avg      0.70      0.69      0.68     19440  
  
{  
    "columns": [  
        {"name": "index", "rawType": "int64", "type": "integer"},  
        {"name": "feature", "rawType": "str", "type": "string"},  
        {"name": "mean", "rawType": "float64", "type": "float"},  
        {"name": "std. dev", "rawType": "float64", "type": "float"}  
    ], "ref": "13c3b663-ce75-4008-adbe-64ae869ae2f9", "rows":  
    [  
        ["0", "frontal_alpha", "0.06451552244690972", "0.06451552244690972"],  
        ["1", "frontal_gamma", "0.07664817459303505", "0.07664817459303505"],  
        ["2", "parietal_alpha", "0.06562660527449811", "0.06562660527449811"],  
    ]  
}
```

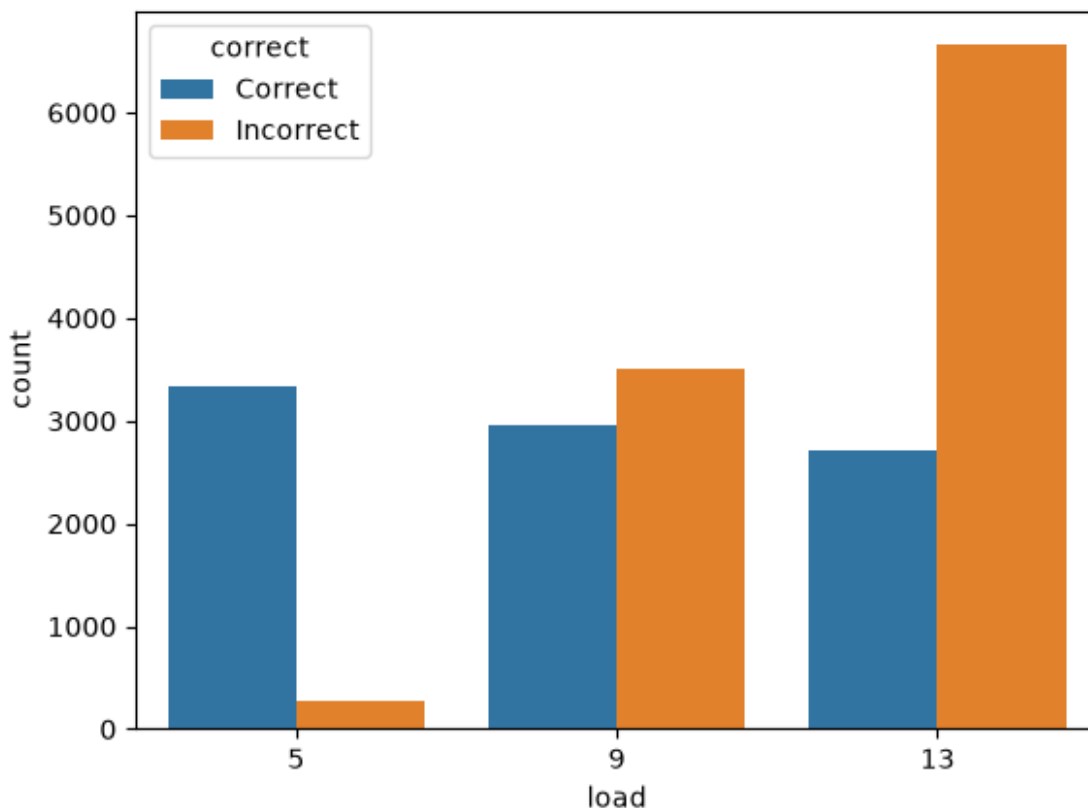


```
[
    ["3", "parietal_gamma", "0.07565699169306653", "0.07565699169306653"],
    ["4", "temporal_alpha", "0.06392659721370081", "0.06392659721370081"],
    ["5", "temporal_gamma", "0.07039143358769781", "0.07039143358769781"],
    ["6", "midline_alpha", "0.06387030336697123", "0.06387030336697123"],
    ["7", "midline_gamma", "0.07069531651362118", "0.07069531651362118"],
    ["8", "load", "0.4486690553104996", "0.4486690553104996"]], "shape":
{"columns": 3, "rows": 9}]
```

Markdown conclusion

At first glance, it seems the model is performing well, with accuracy of 68.9%. However, if we analyse further and look at the feature importance table, we see that the classification is dominated very heavily by load itself. The EEG powers contribute very little to the classification. Furthermore, the bar chart below shows how correct/incorrect drops heavily with load, so the correctness can be predicted almost entirely by load alone. Therefore, this is not a useful use case for EEG analysis.

```
fig, ax = plt.subplots()
sns.countplot(x="load", hue="correct", data=memory_only)
<Axes: xlabel='load', ylabel='count'>
```



Classify: Determine condition from EEG (without baseline)

```
best_params = random_search.best_params_
accuracies, true, predicted = [], [], []
feature_importances = []

for train, test in logit.split(X, Y, groups=group):
    classifier = RandomForestClassifier(**best_params,
    random_state=42, class_weight="balanced")
    classifier.fit(X.iloc[train], Y.iloc[train])
    prediction = classifier.predict(X.iloc[test])
    accuracy = accuracy_score(Y.iloc[test], prediction)
    accuracies.append(accuracy)
    true.extend(Y.iloc[test])
    predicted.extend(prediction)
    feature_importances.append(classifier.feature_importances_)

mean = np.mean(feature_importances, axis=0)
stddev = np.std(feature_importances, axis=0)

print(f"Mean accuracy: {np.mean(accuracies)}")
print(f"Classification report: {classification_report(true,
predicted)}")
feature_importances_df = pd.DataFrame({
    "feature" : X.columns,
    "mean" : mean,
    "std. dev" : stddev
})

display(feature_importances_df)
```

Mean accuracy: 0.6885802469135802

Classification report:

	precision	recall	f1-score	support
Correct	0.75	0.50	0.60	8999
Incorrect	0.66	0.85	0.75	10441
accuracy		0.69		19440
macro avg	0.70	0.68	0.67	19440
weighted avg	0.70	0.69	0.68	19440

```
{
  "columns": [
    {"name": "index", "rawType": "int64", "type": "integer"},
    {"name": "feature", "rawType": "str", "type": "string"},
    {"name": "mean", "rawType": "float64", "type": "float"},
    {"name": "std. dev", "rawType": "float64", "type": "float"}
  ],
  "ref": "13c3b663-ce75-4008-adbe-64ae869ae2f9",
  "rows": [
    ["0", "frontal_alpha", "0.06451552244690972", "0.06451552244690972"],
    ["1", "frontal_gamma", "0.07664817459303505", "0.07664817459303505"],
    ["2", "parietal_alpha", "0.06562660527449811", "0.06562660527449811"],
  ]
}
```

```
[
    ["3", "parietal_gamma", "0.07565699169306653", "0.07565699169306653"],
    ["4", "temporal_alpha", "0.06392659721370081", "0.06392659721370081"],
    ["5", "temporal_gamma", "0.07039143358769781", "0.07039143358769781"],
    ["6", "midline_alpha", "0.06387030336697123", "0.06387030336697123"],
    ["7", "midline_gamma", "0.07069531651362118", "0.07069531651362118"],
    ["8", "load", "0.4486690553104996", "0.4486690553104996"]], "shape":
{"columns": 3, "rows": 9}}
```

I was concerned that applying a baseline to the brain wave power was normalizing our values, leading to poor performance. That is why I tried a model without baseline. As we see, that wasn't a good move.

Compute powers without baseline

```
## LABEL = the event string like x500913
## CODE = a numeric code of the event

### SAME AS RANDOM FOREST 1
def parse_label(label):
    if not label.startswith(("5", "6")) or len(label) > 7 or
len(label) < 6:
        print(f"Skipping this label: label {label} is invalid.")
        return None
    condition_flag = label[0]
    if condition_flag not in ("5", "6"):
        print(f"Error: label {label}. Condition flag is invalid")
    condition = "Memory" if condition_flag == "6" else "Listen"
    position = int(label[2:4])
    load = int(label[4:6])
    correct = None
    if condition == "Memory" and len(label) > 6:
        correct = "Correct" if label[6] == "1" else "Incorrect"
    return {"label": label, "condition": condition, "position":
position, "load": load, "correct": correct}

#####
#####
# CHANNELS ARE: The channel names are: ['Fp1', 'Fz', 'F3', 'F7',
'FT9',
# 'FC5', 'FC1', 'C3', 'T7', 'TP9', 'CP5', 'CP1', 'Pz', 'P3', 'P7',
# 'O1', 'Oz', 'O2', 'P4', 'P8', 'TP10', 'CP6', 'CP2', 'Cz', 'C4',
# 'T8', 'FT10', 'FC6', 'FC2', 'F4', 'F8', 'Fp2', 'AF7', 'AF3', 'AFz',
# 'F1', 'F5', 'FT7', 'FC3', 'C1', 'C5', 'TP7', 'CP3', 'P1', 'P5',
'P07',
# 'P03', 'P0z', 'P04', 'P08', 'P6', 'P2', 'CPz', 'CP4', 'TP8', 'C6',
'C2',
# 'FC4', 'FT8', 'F6', 'AF8', 'AF4', 'F2']

# Groups defined as per: https://neuroanalyzer.org/tutorials/locs.html
roi_groups = {
```

```

    "frontal" : ["Fp1", "Fp2", "F3", "F4", "F7", "F8", "Fz"], #
    Frontal lobe: Executive function, attention, decision-making, and
    working memory.
    "parietal" : ["P3", "P4", "P7", "P8", "Pz", "CP1", "CP2", "CP5",
    "CP6"], # Parietal lobe: Spatial orientation, sensory integration, and
    attention.
    "temporal": ["T7", "T8", "F7", "F8", "FT9", "FT10"], # Temporal
    lobe: Auditory processing, language, and memory.
    "midline" : ["Fz", "Cz", "Pz", "Oz"], # Midline structures:
    Default mode network, global brain states.
}

```

```

# Thus we will have:
# frontal_alpha, frontal_gamma
# parietal_alpha, parietal_gamma
# temporal_alpha, temporal_gamma
# midline_alpha, midline_gamma

```

```

def compute_power(epochs):
    freq = np.concatenate([np.arange(8, 13, 2), np.arange(30, 45, 2)])
    # Alpha and Gamma. We are using 45 because we cutoff after that in our
    # filter. Sparser sampling.
    n_cycles = freq / 2
    powers = epochs.compute_tfr(method="morlet", freqs=freq,
    n_cycles=n_cycles, decim=4, average=False)

```

```

#### DO NOT DOWNSIZE!
## powers.data = powers.data.astype(np.float32)
###

```

```

####
### DO NOT APPLY BASELINE!
###

```

```

alpha = powers.copy().crop(fmin=8, fmax=12)
gamma = powers.copy().crop(fmin=30, fmax=44)

```

```

# The key difference here: get ROI specific channels
# I can do list construction one-line but I like this more
ch_indices = []
roi_powers = {}
for roi_name, ch_list in roi_groups.items():
    ch_indices = []
    for ch in ch_list:
        if ch in powers.ch_names:
            ch_indices.append(powers.ch_names.index(ch))
    roi_powers[f"{roi_name}_alpha"] = alpha.data[:, ch_indices, :,
:] .mean(axis=(1, 2, 3))
    roi_powers[f"{roi_name}_gamma"] = gamma.data[:, ch_indices, :,
:] .mean(axis=(1, 2, 3))

```

```

        return pd.DataFrame(roi_powers)
#####
#####

### SAME AS RANDOM FOREST 1
preprocessed_eeg_dir = "data/ica-excluded-eeg"
def process_participant_data(num):
    print(f"Processing epoch for participant {num}")
    eeg_file = f"{preprocessed_eeg_dir}/sub-0{num}-task-eeg_ica-
cleaned.fif"
    if not os.path.exists(eeg_file):
        print(f"File {eeg_file} not found")
    else:
        raw = mne.io.read_raw_fif(eeg_file, preload=True)
        print(f"Read file {eeg_file}")
        events, event_id = mne.events_from_annotations(raw)
        code_to_label = {v : k for k, v in event_id.items()} # Reverse
map. Get a dict where event "code" is KEY and "label" is VALUE

        # Loop through events in time series
        metadata, valid_event_indices = [], []
        for index, event in enumerate(events):
            event_description = parse_label(code_to_label[event[2]]) # Get
the label from the event code which is in the 3rd position in "event"
            if event_description:
                metadata.append(event_description)
                valid_event_indices.append(index)

        metadata_df = pd.DataFrame(metadata) # Create metadata dataframe
        valid_events = events[valid_event_indices] # Get the list of clean
event codes
        matched_event_labels = {k: v for k, v in event_id.items() if
parse_label(k)} # Get the dictionary of event labels to codes, but
only where it's valid label

        # Create epoch object: from 1.5 before to 3.5 as authors did
        epochs = mne.Epochs(raw, events=valid_events,
event_id=matched_event_labels, tmin=-1.5, tmax=3.5,
metadata=metadata_df)

        print(f"Start computing power for participant {num}")

        powers = compute_power(epochs)
        participant_df = epochs.metadata.reset_index(drop=True).copy()

        # Thus we will have:
        # frontal_alpha, frontal_gamma
        # parietal_alpha, parietal_gamma
        # temporal_alpha, temporal_gamma

```

```

# midline_alpha, midline_gamma

participant_df["frontal_alpha"] = powers["frontal_alpha"].values
participant_df["frontal_gamma"] = powers["frontal_gamma"].values

participant_df["parietal_alpha"] = powers["parietal_alpha"].values
participant_df["parietal_gamma"] = powers["parietal_gamma"].values

participant_df["temporal_alpha"] = powers["temporal_alpha"].values
participant_df["temporal_gamma"] = powers["temporal_gamma"].values

participant_df["midline_alpha"] = powers["midline_alpha"].values
participant_df["midline_gamma"] = powers["midline_gamma"].values

participant_df["participant"] = num

print(f"\nBelow is dataframe for participant {num}")
display(participant_df)
return participant_df

participants_batch_1 = [32, 33, 34, 35, 36, 38, 39, 40, 41, 42]
participants_batch_2 = [43, 44, 45, 46, 47, 48, 49, 50, 51, 52]
participants_batch_3 = [53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63,
64, 65] # We will leave this out for lack of time

all_participants_3 = []

print(f"Starting batch_1. all_participants shape:
{len(all_participants_3)}")
for participant in participants_batch_1:
    print(f"\n\nProcessing epoch and power for participant
{participant}")
    # try:
    all_participants_3.append(process_participant_data(participant))
    # except Exception as error:
    #     print(f"Skipped participant {participant}. Error {error}.")

print(f"Starting batch_2. all_participants shape:
{len(all_participants_3)}")
for participant in participants_batch_2:
    print(f"\n\nProcessing epoch and power for participant
{participant}")
    try:

all_participants_3.append(process_participant_data(participant))
    except Exception as error:
        print(f"Skipped participant {participant}. Error {error}.")

all_participants_df_3 = pd.concat(all_participants_3,
ignore_index=True)

```


support				
Listen	0.36	0.47	0.41	9747
Memory	0.68	0.58	0.63	19440
accuracy			0.54	29187
macro avg	0.52	0.52	0.52	29187
weighted avg	0.57	0.54	0.55	29187

Run random forest

```

logo = LeaveOneGroupOut()
accuracies, true, predicted = [], [], []

for train, test in logo.split(X, Y, groups=group):
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X.iloc[train])
    X_test_scaled = scaler.transform(X.iloc[test])
    classifier = LogisticRegression(max_iter=1000, random_state=42,
class_weight="balanced")
    classifier.fit(X_train_scaled, Y.iloc[train])
    prediction = classifier.predict(X_test_scaled)
    accuracy = accuracy_score(Y.iloc[test], prediction)
    accuracies.append(accuracy)
    true.extend(Y.iloc[test])
    predicted.extend(prediction)

print(f"Mean accuracy: {np.mean(accuracies)}")
print(f"Classification report: {classification_report(true,
predicted)}")

```

Mean accuracy: 0.48295236313754836

Classification report: precision recall f1-score

support				
Listen	0.35	0.65	0.46	9747
Memory	0.69	0.40	0.51	19440
accuracy			0.48	29187
macro avg	0.52	0.52	0.48	29187
weighted avg	0.58	0.48	0.49	29187

Markdown Conclusion

This model performs worse than chance, making multiple wrong classifications. As a result, we cannot reliably use this model. See below for a diagnosis on why this model fails....

Diagnosing model performance

```
df_features_3["condition"].isna().sum()
display(X.describe())
print(X.isna().sum())

{"columns":[{"name":"index","rawType":"str","type":"string"},
{"name":"frontal_alpha","rawType":"float64","type":"float"},
{"name":"frontal_gamma","rawType":"float64","type":"float"},
{"name":"parietal_alpha","rawType":"float64","type":"float"},
{"name":"parietal_gamma","rawType":"float64","type":"float"},
{"name":"temporal_alpha","rawType":"float64","type":"float"},
{"name":"temporal_gamma","rawType":"float64","type":"float"},
{"name":"midline_alpha","rawType":"float64","type":"float"},
{"name":"midline_gamma","rawType":"float64","type":"float"}],"ref":"62
67ee45-4222-4830-819d-814d000c7c8b","rows":
[["count","29187.0","29187.0","29187.0","29187.0","29187.0","29187.0",
"29187.0","29187.0"],["mean","2.097507394692095e-
09","1.2361981365502639e-10","3.100704022846159e-
09","1.106547679511473e-10","1.520537594476293e-
09","1.1180742414664877e-10","2.886852586864658e-
09","1.1985778791970822e-10"],["std","2.4327358428413093e-
09","1.310786206477997e-10","5.080435717230288e-
09","1.0845133791077701e-10","1.787225865215688e-
09","1.7733897942476421e-10","3.949038978203699e-
09","9.48159499155036e-11"],["min","1.3611582112432822e-
10","1.4297621181707508e-11","2.2876583781217548e-
10","2.0660955799874368e-11","1.2673024754078903e-
10","1.3239262084213457e-11","1.9945360727028823e-
10","2.3464241047027132e-11"],["25%","6.955114383081233e-
10","7.051348673803591e-11","8.519479182093357e-
10","6.393431740885385e-11","5.378647678829892e-
10","5.924700927388826e-11","9.361776567133665e-
10","6.691291086734648e-11"],["50%","1.2573872834796703e-
09","9.335942614097158e-11","1.504076235030799e-
09","8.720711255271583e-11","8.317296745251676e-
10","8.022963887564426e-11","1.5944004490998244e-
09","9.433352855285656e-11"],["75%","2.4251748123415766e-
09","1.355212772406614e-10","2.9874927962316208e-
09","1.277123248227938e-10","1.6537573737113722e-
09","1.2000693090270945e-10","2.9367914499340263e-
09","1.336931042197577e-10"],["max","1.8584898143388137e-
08","3.9951239562585595e-09","4.3260598787469144e-
08","3.825225589365025e-09","1.5609776792162712e-
08","9.513608038846748e-09","2.87511678329918e-
08","1.3863090130659243e-09"]],"shape":{"columns":8,"rows":8}}
```

frontal_alpha	0
frontal_gamma	0
parietal_alpha	0

```
parietal_gamma    0
temporal_alpha    0
temporal_gamma    0
midline_alpha     0
midline_gamma     0
dtype: int64
```

We first see if there are any strange or NAN values. There aren't but there is a reason that our model is still performing poorly. After we remove the baseline, we remove any concept of increase or decrease in power. This makes our model perform worse.

Conclusions, discussion, and next steps

- We need more biomarkers like ECG and Pupillometry (like the authors did)*
- Potentially explore ERP analysis
- Compare change in power relative to participant baseline
- EEG power alone cannot reliably predict between Memory and Listen in this setup